Python cheatsheet · 19 chapters · only Python code

# Playwright with Python: cheatsheet

The key ideas, every example program in Python, the tables and the common mistakes from every chapter, in one place for quick revision. Everything here comes from the chapters and shows Python only, so this sheet is all you need. Each card links back to the lesson that teaches it.

95 key ideas | 97 code examples | 77 tables | 73 tips and traps

## STEP 1 Getting started

### 1.1 What Playwright is and how it works

What Playwright is and the jobs teams use it for, the official languages, a fact based comparison with Selenium and Cypress, a first script in four languages, how one call travels through the client library, the driver and the browser protocol with real DEBUG=pw:api, pw:channel and pw:protocol logs, a first test with JUnit, pytest and Playwright Test, a recorded sign in walk, and what a test runner does for every test.

KEY IDEAS

- **You write steps** Open, type, click, check. In Java, Python, JavaScript or TypeScript.
- **Messages** Each call becomes a small message. The driver is the same for every language.
- **Real browsers** Real clicks and real typing. The page cannot tell it apart from a person.
- **It waits** Visible, stable, enabled. No fixed sleeps, so far fewer flaky tests.
- **Remember** Playwright drives real browsers through one driver, waits for elements before it acts, and works the same way in Java, Python, JavaScript and TypeScript.

CODE

**Your first Playwright script**

first.py

```python
from playwright.sync_api import sync_playwright

with sync_playwright() as p:                        # start the Playwright driver
    browser = p.chromium.launch()                   # start a real Chromium, with no window
    page = browser.new_page()                       # open a new tab
    page.goto("http://localhost:4173/products")
    print("Title:", page.title())
    page.screenshot(path="products.png")            # save a picture of the page
    browser.close()
```

Run it: `python first.py && ls *.png`

Lesson: Your first script

**Watch what Playwright does inside each call**

first.py

```python
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()
    page.goto("http://localhost:4173/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_role("button", name="Sign in").click()
    browser.close()
```

Run it: `DEBUG=pw:api python first.py`

Lesson: How one line travels to the browser

### Your first test, in four languages

tests/test_sign_in.py

```python
from playwright.sync_api import Page, expect


def test_a_customer_can_sign_in(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()
```

Run it: `pytest -v`

Lesson: From a script to a test

### A customer signs in

| | |
|---|---|
| `page.goto("http://localhost:4173/login")` | Open the sign in page. |
| `page.get_by_label("Email").fill("asha@shop.example")` | Type the email into the box labelled Email. |
| `page.get_by_label("Password").fill("s3cret")` | Type the password. |
| `page.get_by_role("button", name="Sign in").click()` | Click the button called Sign in. The shop signs Asha in and opens the products page. |
| `expect(page.get_by_text("Hello, Asha")).to_be_visible()` | Check that the greeting says Hello, Asha. The check waits until it appears. |

Lesson: From a script to a test

## TABLES

### Table: language · package · usual test runner · version used on this page

| Language | Package | Usual test runner | Version used on this page |
|---|---|---|---|
| 🐍 Python | `playwright` and `pytest-playwright` on PyPI | pytest | 1.63.0 |

Lesson: What Playwright is

### Table: playwright · selenium webdriver · cypress

| | Playwright | Selenium WebDriver | Cypress |
|---|---|---|---|
| Languages | Java, Python, JavaScript, TypeScript, C# | Java, Python, JavaScript, C#, Ruby | JavaScript and TypeScript |
| Browsers | Chromium, Firefox, WebKit | Chrome, Firefox, Edge, Safari through their drivers | Chromium family, Firefox, WebKit (experimental) |
| How it talks to the browser | Browser debugging protocols, through one driver process | The W3C WebDriver protocol, through a driver per browser | Runs inside the browser next to your app |
| Waiting | Automatic before every action and in every check | You add waits yourself | Automatic retries for commands and checks |
| Several tabs, windows and domains in one test | Yes | Yes | Limited |
| Parallel runs | Built in, free | With Selenium Grid or a test runner | Paid cloud service, or plugins |
| Debugging | Trace viewer, UI mode, inspector, codegen | Logs and screenshots you add | Time travel in the runner |

Lesson: Why teams pick Playwright

**Table: job · 🐍 python**

| Job | 🐍 Python |
|---|---|
| Run the tests | `pytest` |
| Where settings live | `pytest.ini` and `conftest.py` |
| A fresh page for each test | the `page` fixture |
| Checks that wait | `expect(...)` |
| Run in parallel | `pytest -n 4` with pytest-xdist |
| HTML report | pytest-html or Allure |

Lesson: What the test runner does for you

**Table: module · you will learn**

| Module | You will learn |
|---|---|
| 🚀 Getting started | what Playwright is, how it works inside, installing it in four languages, codegen and VS Code |
| 🌐 Browser handling | browsers, contexts, pages, devices, tabs, pop ups, frames, dialogs, downloads and uploads |
| 🎯 Finding and using elements | locators, actions and auto waiting, and checks that wait |
| 🏃 The test runner | configuration, tags, data driven tests, fixtures, hooks, parallel runs, retries and timeouts |
| 🔌 Network and API | mocking the server, recording traffic, API tests and signing in once |
| 🐞 Debugging and quality | trace viewer, UI mode, screenshots, videos, visual and accessibility tests, the clock |
| 🏗️ Framework design | page objects, a real company framework built step by step in each language, CI and Docker |
| 📊 Reporting labs | the built in reporters, HTML report, Allure, pytest-html, ExtentReports and your own reporter |
| 🆕 New and AI features | what changed from version 1.45 to 1.64, the Playwright MCP server and test agents |

Lesson: What this subject covers

### TIPS AND TRAPS

COMMON MISTAKE People often ask for Playwright in C, C++ or Go. There is no official Playwright for C or C++. For Go there is a community project, playwright-go, which is not made by the Playwright team. This subject sticks to the official languages so that every example is supported and up to date.

Lesson: What Playwright is

IN REAL TEAMS In many companies you will join a project that already uses Selenium. Knowing both is useful. Many teams move to Playwright one test suite at a time, because the ideas are close: find an element, act on it, check the result.

Lesson: Why teams pick Playwright

REMEMBER The driver is the same Node.js program for every language. This is why a new feature arrives in Java, Python, JavaScript and C# in the same release.

Lesson: How one line travels to the browser

REMEMBER The test asks for a `page` argument. pytest-playwright sees the name and gives the test a fresh page. The `--base-url` option in pytest.ini makes `/login` mean http://localhost:4173/login.

Lesson: From a script to a test

## 1.2 Installing Playwright and your first project

Runtime and operating system requirements, installing with Maven, pip and npm init playwright, checking the version, where browsers are installed and Chrome for Testing, the real files of a scaffolded project and typical Python and Java layouts, codegen with real screenshots and the recorded code in four languages, cleaning up recorded code, running, filtering and listing tests, editors and updating safely.

## KEY IDEAS

- **Runtime** Install your language first. Check it with a version command.
- **Package** Pin the exact version.
- **Browsers** Playwright brings its own. --with-deps adds the Linux libraries.
- **Record** Click around, get code. Then clean it up and add a check.
- **Remember** Install the runtime, the Playwright package and its browsers, record a first draft with codegen, then turn it into a real test with a clear name and a check.

## CODE

### Check what is installed

Run it: `pip list 2>/dev/null | grep -i playwright && python -m playwright --version`

Lesson: Install Playwright in your language

### Where the browsers live

Run it: `playwright install --dry-run chromium`

Lesson: Install Playwright in your language

### What codegen recorded, run as it is

tests/test_example.py

```python
import pytest
import re
from playwright.sync_api import Page, expect


@pytest.fixture(scope="session")
def browser_context_args(browser_context_args, playwright):
    return {"viewport": {"width":800,"height":600}}


def test_example(page: Page) -> None:
    page.goto("http://localhost:4173/login")
    page.get_by_role("textbox", name="Email").click()
    page.get_by_role("textbox", name="Email").fill("asha@shop.example")
    page.get_by_role("textbox", name="Password").click()
    page.get_by_role("textbox", name="Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
```

Run it: `pytest -v`

Lesson: Record a test with codegen

### The same test after a clean up

tests/test_sign_in.py

```python
from playwright.sync_api import Page, expect


def test_a_customer_can_sign_in(page: Page):
    page.goto("/login")
    page.get_by_role("textbox", name="Email").fill("asha@shop.example")
    page.get_by_role("textbox", name="Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()
```

Run it: `pytest -v`

Lesson: Record a test with codegen

### Run only one test from a file

tests/test_shop.py

```python
from playwright.sync_api import Page, expect


def test_a_customer_can_sign_in(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
```

```
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()


def test_the_products_page_has_a_title(page: Page):
    page.goto("/products")
    expect(page).to_have_title("Products · Brainy Shop")
```

Run it: `pytest -v -k sign_in`

Lesson: Running tests from the terminal

### List the tests without running them

tests/test_shop.py

```
from playwright.sync_api import Page, expect


def test_a_customer_can_sign_in(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()


def test_the_products_page_has_a_title(page: Page):
    page.goto("/products")
    expect(page).to_have_title("Products · Brainy Shop")
```

Run it: `pytest --collect-only -q`

Lesson: Running tests from the terminal

## TABLES

### Table: 🐍 python

| | 🐍 Python |
|---|---|
| Runtime | Python 3.8 or later (this page uses 3.13, and the newest pytest needs 3.10) |
| Package tool | pip, Poetry or uv |
| Editor most teams use | PyCharm or VS Code |
| Operating system | same |

Lesson: What you need before you start

### Table: i want to · 🐍 python (pytest)

| I want to | 🐍 Python (pytest) |
|---|---|
| run every test | `pytest` |
| run one file | `pytest tests/test_shop.py` |
| run tests by name | `-k sign_in` |
| see the browser | `--headed` |
| slow every action down | `--slowmo 500` |
| pick a browser | `--browser firefox` |
| debug step by step | `PWDEBUG=1 pytest -s` |
| run 4 at a time | `-n 4` (pytest-xdist) |

Lesson: Running tests from the terminal

**Table: language · update the package · then**

| Language | Update the package | Then |
|---|---|---|
| 🐍 Python | `pip install -U pytest-playwright playwright` | `playwright install --with-deps` |

Lesson: Updating Playwright safely

### TIPS AND TRAPS

COMMON MISTAKE Keep one Playwright version per project and update on purpose. Each Playwright release expects its own browser builds. After you update the package, run the install command again, or the tests stop with an error that says the browser executable does not exist.

Lesson: Install Playwright in your language

IN REAL TEAMS Codegen picks locators the same way you should: role and accessible name first, then label, placeholder or text, and a test id only when nothing better exists. When the recorded locator looks odd, the page itself is often missing a label, which is worth reporting to the developers.

Lesson: Record a test with codegen

IN REAL TEAMS Turn on format on save and a linter in your editor from day one.

Lesson: Your editor

REMEMBER Pin exact versions in your project (no ^ in package.json, == in requirements.txt). Then a teammate and the CI server always run the same Playwright, and an update is a change you can see and review.

Lesson: Updating Playwright safely

## STEP 2 Browser handling

### 2.1 Browsers, contexts, pages and devices

The browser, context and page levels with an interactive builder, launch options such as headless, slowMo and channel, two signed in users in one browser, context options for viewport, locale, time zone, colour scheme, touch, geolocation and permissions, device profiles, new tabs and popups, Chromium, Firefox and WebKit projects, persistent profiles, connectOverCDP and browser servers.

### KEY IDEAS

- **Browser** Start once, about 0.5 s. Shared by many tests.
- **Context** A fresh, empty profile. Its own cookies, storage and permissions.
- **Page** One tab. Tabs in a context share the login.
- **Emulation** Size, locale, time zone, location. Set on the context, nothing on your computer changes.
- **Remember** Start one browser, give every test a new context, and set what the site should see, such as a phone or a German user, with context options.

### CODE

**Launch a browser with options**

main.py

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(
        headless=True,      # no window. Use False to watch it work
        slow_mo=100,        # wait 100 ms before every action
        args=["--lang=en-GB"],
    )
    print("Browser type:", browser.browser_type.name)
    print("Version:", browser.version)
```

```
    print("Connected:", browser.is_connected())
    print("Contexts so far:", len(browser.contexts))
    browser.close()
    print("Connected after close:", browser.is_connected())
```

Run it: `python main.py`

Lesson: Launching a browser

### Two shoppers in one browser, kept apart

tests/test_contexts.py

```
from playwright.sync_api import expect


def test_two_shoppers_in_one_browser(new_context):
    # Each context is a separate, empty browser profile.
    asha_context = new_context()
    guest_context = new_context()
    asha = asha_context.new_page()
    guest = guest_context.new_page()

    asha.goto("/login")
    asha.get_by_label("Email").fill("asha@shop.example")
    asha.get_by_label("Password").fill("s3cret")
    asha.get_by_role("button", name="Sign in").click()
    expect(asha.get_by_text("Hello, Asha")).to_be_visible()

    guest.goto("/products")
    expect(guest.get_by_role("link", name="Sign in")).to_be_visible()
    print("Asha cookies:", len(asha_context.cookies()))
    print("Guest cookies:", len(guest_context.cookies()))
```

Run it: `pytest -v -s`

Lesson: Contexts keep tests apart

### Pretend to be a German phone user in India

tests/test_emulation.py

```
import pytest
from playwright.sync_api import Page, expect


@pytest.mark.browser_context_args(
    viewport={"width": 390, "height": 844},
    locale="de-DE",
    timezone_id="Asia/Kolkata",
    color_scheme="dark",
    has_touch=True,
)
def test_the_shop_sees_a_german_phone_user_in_india(page: Page):
    page.goto("/about-you")
    for id in ["width", "lang", "tz", "price", "touch"]:
        print(id.ljust(6), page.locator("#" + id).text_content())
    dark = page.evaluate("matchMedia('(prefers-color-scheme: dark)').matches")
    print("dark  ", dark)
    expect(page.locator("#lang")).to_have_text("de-DE")
```

Run it: `pytest -v -s`

Lesson: Pretend to be another user

### Location and permissions

tests/test_location.py

```
import pytest
from playwright.sync_api import Page, expect


@pytest.mark.browser_context_args(
    geolocation={"latitude": 28.6139, "longitude": 77.209},    # New Delhi
    permissions=["geolocation"],
)
def test_finds_stores_near_new_delhi(page: Page):
    page.goto("/about-you")
```

```
    page.get_by_role("button", name="Find stores near me").click()
    expect(page.locator("#where")).to_have_text("28.6139, 77.2090")


def test_location_blocked_when_permission_is_not_granted(page: Page):
    page.goto("/about-you")
    page.get_by_role("button", name="Find stores near me").click()
    expect(page.locator("#where")).to_contain_text("blocked")
```

Run it: `pytest -v`

Lesson: Pretend to be another user

### Emulate a phone from the device list

tests/test_device.py

```
from playwright.sync_api import Page, expect


def test_the_shop_on_a_pixel_7(page: Page):
    page.goto("/about-you")
    print("width ", page.locator("#width").text_content())
    print("touch ", page.locator("#touch").text_content())
    print("mobile", page.evaluate("/Mobile/.test(navigator.userAgent)"))
    expect(page.locator("#touch")).to_have_text("yes")
```

Run it: `pytest -v -s --device "Pixel 7"`

Lesson: Phones and tablets

### New tabs and pop up windows

tests/test_tabs.py

```
from playwright.sync_api import Page, expect


def test_help_opens_in_a_new_tab(page: Page):
    page.goto("/widgets")
    # Start waiting for the new tab before the click that opens it.
    with page.expect_popup() as popup_info:
        page.get_by_role("link", name="Open help in a new tab").click()
    help = popup_info.value

    expect(help).to_have_title("Help · Brainy Shop")
    print("Tabs open:", len(page.context.pages))
    help.close()
    print("Tabs open:", len(page.context.pages))


def test_the_chat_window_is_a_popup(page: Page):
    page.goto("/widgets")
    with page.expect_popup() as popup_info:
        page.get_by_role("button", name="Open chat window").click()
    expect(popup_info.value.get_by_role("heading")).to_have_text("Chat with us")
```

Run it: `pytest -v -s`

Lesson: New tabs and pop ups

### A persistent profile that survives a restart

main.py

```
import tempfile
from playwright.sync_api import sync_playwright

profile = tempfile.mkdtemp(prefix="shop-profile-")

with sync_playwright() as p:
    # First visit: a real profile folder on disk, like a person's Chrome profile.
    context = p.chromium.launch_persistent_context(profile)
    page = context.pages[0]
    page.goto("http://localhost:4173/")
    page.evaluate("localStorage.setItem('theme', 'dark')")
    context.close()

    # Second visit with the same folder: the data is still there.
```

```
    context = p.chromium.launch_persistent_context(profile)
    page = context.pages[0]
    page.goto("http://localhost:4173/")
    print("theme =", page.evaluate("localStorage.getItem('theme')"))
    context.close()
```

Run it: `python main.py`

Lesson: Profiles, closing and remote browsers

## TABLES

### Table: option · what it does · when you use it

| Option | What it does | When you use it |
|---|---|---|
| headless | run without a window (the default) | always on CI, turn off to watch a test locally |
| slowMo | pause before each action | to follow a test with your eyes or record a demo |
| channel | use an installed branded browser: `chrome`, `msedge`, `chrome-beta` | when the team must test the exact browser users have |
| args | extra command line switches for the browser | rare cases, such as a language or a fake camera |
| executablePath | start a browser from a path you give | company images with their own browser build |
| timeout | how long to wait for the browser to start | slow CI machines |

Lesson: Launching a browser

### Table: option · example · what the page sees

| Option | Example | What the page sees |
|---|---|---|
| viewport | 390 × 844 | `innerWidth`, media queries |
| locale | de-DE | `navigator.language`, number and date formats |
| timezoneId | Asia/Kolkata | dates and times |
| colorScheme | dark | `prefers-color-scheme` |
| hasTouch, isMobile | true | touch events, mobile layout |
| geolocation, permissions | New Delhi | the location API |
| userAgent | any string | `navigator.userAgent` |
| offline | true | no network at all |

Lesson: Pretend to be another user

### Table: 🐍 python

| | 🐍 Python |
|---|---|
| pick the engine | `--browser firefox` |
| all three in one run | `--browser chromium --browser firefox --browser webkit` |
| branded Chrome or Edge | `--browser-channel chrome` |

Lesson: Chromium, Firefox and WebKit

## TIPS AND TRAPS

IN REAL TEAMS Real projects use two contexts in one test for features with two people: a chat between a customer and a support agent, an admin approving an order while the customer waits, or two players in a game. Each person gets their own context.

Lesson: Contexts keep tests apart

COMMON MISTAKE Emulation catches most layout and touch bugs and costs nothing. Some bugs only show on real hardware, such as a keyboard covering a field or a slow phone. Many teams run emulated phones on every commit and real devices in a cloud service before a release.

Lesson: Phones and tablets

REMEMBER WebKit on Windows and Linux is the engine of Safari, not Safari itself. It catches most Safari bugs. For the last few percent, teams run a small smoke suite on real Safari on a Mac.

Lesson: Chromium, Firefox and WebKit

REMEMBER Close what you open. In a test runner the fixtures close contexts and pages for you. In a script, close the browser at the end, or use try with resources in Java and a with block in Python, so a crash does not leave browser processes running on the machine.

Lesson: Profiles, closing and remote browsers

## 2.2 Navigation, frames, dialogs, files and events

What goto waits for, waiting for the URL after a click, back, forward and reload, a recorded walk through a payment iframe, frameLocator and contentFrame, alert, confirm and prompt with the default dismiss, downloads with suggestedFilename and saveAs, uploads from memory, console, pageerror and request events, evaluate and addInitScript, and locating inside shadow DOM.

### KEY IDEAS

- **Navigation** goto waits for load. After a click, wait for the new URL.
- **Iframes** Step in first. Then locate as usual.
- **Dialogs** Dismissed by default. Handle them before the click.
- **Files** Wait for downloads. Fill file inputs directly.
- **Remember** Start waiting before the click for anything the click creates: a new URL, a popup, a dialog or a download.

### CODE

**Go, wait for the address, go back and reload**

tests/test_navigation.py

```
import re
from playwright.sync_api import Page, expect


def test_moving_around_the_shop(page: Page):
    page.goto("/login", wait_until="domcontentloaded")
    print("1.", page.url)

    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    page.wait_for_url("**/products")              # the app moves on by itself
    print("2.", page.url)

    page.go_back()
    expect(page).to_have_url(re.compile(r"/login$"))
    print("3.", page.url)

    page.go_forward()
    page.reload()
    expect(page).to_have_url(re.compile(r"/products$"))
    print("4.", page.url, "|", page.title())
```

Run it: `pytest -v -s`

Lesson: Going to pages and waiting for them

### Paying inside an iframe

| | |
|---|---|
| `page.goto("http://localhost:4173/frames")` | Open the checkout page. The card form is an iframe from the payment company. |
| `page.frame_locator('iframe[title="Payment form"]').get_by_label("Card number").fill("4111 1111 1111 1111")` | Step into the iframe and type the card number. |
| `page.frame_locator('iframe[title="Payment form"]').get_by_label("Expiry").fill("12/29")` | Type the expiry date. |
| `page.frame_locator('iframe[title="Payment form"]').get_by_role("button", name="Pay ₹219").click()` | Click Pay inside the iframe. |
| `expect(page.get_by_role("status")).to_have_text("Paid with card ending 1111")` | Back in the main page, the shop shows the result. |

Lesson: Iframes

### Fill a card form that lives inside an iframe

tests/test_frames.py

```python
from playwright.sync_api import Page, expect


def test_pay_inside_the_payment_iframe(page: Page):
    page.goto("/frames")
    payment = page.frame_locator('iframe[title="Payment form"]')

    payment.get_by_label("Card number").fill("4111 1111 1111 1111")
    payment.get_by_label("Expiry").fill("12/29")
    payment.get_by_role("button", name="Pay ₹219").click()

    # The message appears in the main page, outside the iframe.
    expect(page.get_by_role("status")).to_have_text("Paid with card ending 1111")
    print("Frames on the page:", len(page.frames))


def test_a_short_card_number_is_refused(page: Page):
    page.goto("/frames")
    # Another way in: find the iframe element, then step into it.
    payment = page.get_by_title("Payment form").content_frame
    payment.get_by_label("Card number").fill("4111")
    payment.get_by_role("button", name="Pay ₹219").click()
    expect(payment.get_by_role("alert")).to_have_text("Card number must have 16 digits")
```

Run it: `pytest -v -s`

Lesson: Iframes

### Alerts, confirms and prompts

tests/test_dialogs.py

```python
from playwright.sync_api import Page, expect


def test_with_no_handler_dialogs_are_dismissed(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Delete account").click()
    expect(page.get_by_role("status")).to_have_text("Kept your account")


def test_accept_a_confirm_and_read_its_message(page: Page):
    page.goto("/widgets")

    def handle(dialog):
        print(dialog.type, "|", dialog.message)
        dialog.accept()

    page.on("dialog", handle)
    page.get_by_role("button", name="Delete account").click()
    expect(page.get_by_role("status")).to_have_text("Account deleted")
```

```python
def test_type_an_answer_into_a_prompt(page: Page):
    page.goto("/widgets")
    page.once("dialog", lambda dialog: dialog.accept("Weekly shop"))
    page.get_by_role("button", name="Rename list").click()
    expect(page.get_by_role("status")).to_have_text("List renamed to Weekly shop")
```

Run it: `pytest -v -s`

Lesson: Alerts, confirms and prompts

### Catch a download and read the file

tests/test_download.py

```python
from playwright.sync_api import Page


def test_download_the_orders_report(page: Page):
    page.goto("/widgets")
    with page.expect_download() as download_info:
        page.get_by_role("link", name="Download report").click()
    download = download_info.value

    print("File name:", download.suggested_filename)
    text = download.path().read_text()
    print(text.strip())
    assert text.split("\n")[0] == "order,total"
    download.save_as("reports/" + download.suggested_filename)   # keep a copy
```

Run it: `pytest -v -s`

Lesson: Downloads and uploads

### Upload files without touching the disk

tests/test_upload.py

```python
from playwright.sync_api import Page, expect


def test_send_feedback_with_two_screenshots(page: Page):
    page.goto("/forms")
    page.get_by_placeholder("Full name").fill("Asha")
    page.get_by_label("Screenshots").set_input_files([
        {"name": "cart.png", "mimeType": "image/png", "buffer": b"fake picture 1"},
        {"name": "error.png", "mimeType": "image/png", "buffer": b"fake picture 2"},
    ])
    page.get_by_role("button", name="Send feedback").click()
    expect(page.get_by_role("status")).to_contain_text("files: cart.png, error.png")
```

Run it: `pytest -v`

Lesson: Downloads and uploads

### Listen to console messages, page errors and requests

tests/test_events.py

```python
from playwright.sync_api import Page


def test_catch_console_messages_and_page_errors(page: Page):
    messages = []
    page.on("console", lambda msg: messages.append(f"{msg.type}: {msg.text}"))
    page.on("request", lambda request: print("request →", request.method, request.url)
            if "/api/" in request.url else None)
    page.goto("/widgets")

    with page.expect_console_message(lambda msg: msg.type == "warning"):
        page.get_by_role("button", name="Write to the console").click()
    print(messages)

    with page.expect_event("pageerror") as error_info:
        page.get_by_role("button", name="Break the page").click()
    print("Page error:", error_info.value.message)
```

Run it: `pytest -v -s`

Lesson: Listening to what the page does

### Run JavaScript inside the page

tests/test_evaluate.py

```python
from playwright.sync_api import Page, expect


def test_run_code_inside_the_page(page: Page):
    # Runs in the page before any of its own scripts, on every navigation.
    page.add_init_script("window.testMode = true")
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)   # wait for the list

    info = page.evaluate("""() => ({
        title: document.title,
        cards: document.querySelectorAll('li.card').length,
        testMode: window.testMode,
    })""")
    print(info)

    total = page.evaluate("([a, b]) => a + b", [20, 199])   # pass values in
    print("sum =", total)

    tag = page.get_by_placeholder("Search products").evaluate("el => el.tagName")
    print("tag =", tag)
```

Run it: `pytest -v -s`

Lesson: Running code inside the page

### Click a button inside a shadow DOM component

tests/test_shadow.py

```python
from playwright.sync_api import Page, expect


def test_vote_inside_a_shadow_dom_component(page: Page):
    page.goto("/shadow")
    asha_review = page.locator("review-card").filter(has_text="Asha")

    # get_by_role and CSS look inside open shadow roots by themselves.
    asha_review.get_by_role("button", name="Helpful").click()
    expect(asha_review.get_by_text("found this helpful")).to_have_text("1 person found this helpful")

    print("CSS sees the buttons:", page.locator("review-card button").count())
    print("XPath sees the buttons:", page.locator("//review-card//button").count())
```

Run it: `pytest -v -s`

Lesson: Shadow DOM and web components

## TABLES

### Table: dialog · opened by · accept() · dismiss()

| Dialog | Opened by | accept() | dismiss() |
|---|---|---|---|
| alert | `alert(msg)` | closes it | closes it |
| confirm | `confirm(msg)` | the page gets true | the page gets false |
| prompt | `prompt(msg, default)` | the page gets your text | the page gets null |
| beforeunload | leaving a page with unsaved changes | leave the page | stay on the page |

Lesson: Alerts, confirms and prompts

**Table: event · tells you · typical use**

| Event | Tells you | Typical use |
|---|---|---|
| console | a message the page logged | fail on `console.error`, debug a test |
| pageerror | an error nothing caught | fail the test on broken JavaScript |
| request and response | every network call | check an API was called, count calls |
| popup, download, dialog | a new tab, a file, a dialog | handle it in the test |
| close, crash | the page closed or crashed | clean up, report a crash |

Lesson: Listening to what the page does

### TIPS AND TRAPS

COMMON MISTAKE Do not wait for the network to go quiet. A chat widget or analytics script can keep it busy forever. Wait for what the user is waiting for: a URL, a heading or a message. Every check with expect already waits.

Lesson: Going to pages and waiting for them

REMEMBER Some pages hide the real file input and open the file window from a pretty button. Then wait for the file chooser event: `page.expect_file_chooser()` in Python, and call `setFiles` on it.

Lesson: Downloads and uploads

IN REAL TEAMS Many companies add one shared fixture that collects page errors and fails any test that had one.

Lesson: Listening to what the page does

## STEP 3 Finding and using elements

### 3.1 Locators: finding elements

Why a locator is searched again every time, a live locator lab, which locator to try first, getByRole with names, levels and states, labels, placeholders, text, titles and test ids, custom test id attributes, CSS and XPath and why they break, filter, chaining, nth, and, or, strict mode violations, and a recorded walk through a sortable, paged table.

### KEY IDEAS

- **Role and name** What the user sees. The first choice.
- **Forms** Boxes by their label. Placeholder when there is none.
- **Narrow down** Card first, then button. hasText, has, hasNot.
- **One element** Actions need exactly one. The error lists them all.
- **Remember** A locator is searched again every time you use it, and an action needs exactly one match.

### CODE

**A locator is found again every time you use it**

tests/test_lazy.py

```
from playwright.sync_api import Page, expect


def test_a_locator_is_a_recipe_not_an_element(page: Page):
    page.goto("/products")
    cards = page.get_by_role("listitem")          # nothing is searched yet
    expect(cards).to_have_count(5)
    print("before search:", cards.count())

    page.get_by_placeholder("Search products").fill("o")
    print('after "o":', cards.count())            # the same locator, found again

    page.get_by_placeholder("Search products").fill("bottle")
```

```
    expect(cards).to_have_count(1)
    print('after "bottle":', cards.count())
    print(cards.describe("product cards"))
```

Run it: `pytest -v -s`

Lesson: What a locator is

### getByRole with names, levels and states

tests/test_roles.py

```
from playwright.sync_api import Page, expect


def test_find_things_the_way_a_user_sees_them(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)   # waits for the list

    print("headings:", page.get_by_role("heading").all_text_contents())
    print("level 1 heading:", page.get_by_role("heading", level=1).text_content())

    # The name is matched ignoring case, and part of the name is enough.
    print('"add to cart":', page.get_by_role("button", name="add to cart").count())
    # exact=True wants the whole name with the same case.
    print('"add to cart" exact:', page.get_by_role("button", name="add to cart", exact=True).count())
    print("disabled buttons:", page.get_by_role("button", disabled=True).count())

    page.get_by_role("searchbox", name="Search").fill("lamp")
    page.get_by_role("combobox", name="Sort by").select_option("high")
    expect(page.get_by_role("listitem")).to_have_count(1)
    print("left after search:", page.get_by_role("listitem").get_by_role("heading").text_content())
```

Run it: `pytest -v -s`

Lesson: getByRole: the first choice

### Labels, placeholders, text, titles and test ids

tests/test_getby.py

```
import re
from playwright.sync_api import Page, expect


def test_labels_placeholders_text_titles_and_test_ids(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_text("Remember me").click()          # clicking a label ticks its box
    expect(page.get_by_label("Remember me")).to_be_checked()
    page.get_by_role("button", name="Sign in").click()

    expect(page.get_by_text("Hello, Asha")).to_be_visible()
    expect(page.get_by_test_id("cart-count")).to_have_text("0")
    expect(page.get_by_placeholder("Search products")).to_be_editable()
    expect(page.get_by_test_id(re.compile(r"^product-"))).to_have_count(5)   # all product cards

    page.goto("/frames")
    print("iframes by title:", page.get_by_title("Payment form").count())
```

Run it: `pytest -v -s`

Lesson: Labels, placeholders, text, alt text, titles and test ids

### CSS and XPath

tests/test_css.py

```
from playwright.sync_api import Page, expect


def test_css_and_xpath_when_nothing_better_fits(page: Page):
    page.goto("/products")
    expect(page.locator("li.card")).to_have_count(5)

    print("CSS prices:", page.locator("li.card .price").all_text_contents())
    print("CSS attribute:", page.locator('[data-testid="product-3"] h3').text_content())
    print("CSS :has-text():", page.locator('li.card:has-text("mug") h3').text_content())
```

```
    print("XPath:", page.locator('//li[h3="Desk lamp"]//span[@class="price"]').text_content())
    print("id:", page.locator("#sort").input_value())
```

Run it: `pytest -v -s`

Lesson: CSS and XPath

### filter, first, nth, last, and, or

tests/test_filter.py

```
from playwright.sync_api import Page, expect


def test_narrow_down_with_filter_chaining_and_and_or(page: Page):
    page.goto("/products")
    cards = page.get_by_role("listitem")
    expect(cards).to_have_count(5)

    mug = cards.filter(has_text="Coffee mug")
    print("mug price:", mug.locator(".price").text_content())

    kitchen = cards.filter(has_text="Kitchen")
    print("kitchen:", kitchen.get_by_role("heading").all_text_contents())

    sold_out = cards.filter(has=page.get_by_role("button", name="Out of stock"))
    print("sold out:", sold_out.get_by_role("heading").text_content())

    in_stock = cards.filter(has_not=page.get_by_role("button", disabled=True))
    print("in stock:", in_stock.count())

    print("first:", cards.first.get_by_role("heading").text_content())
    print("nth(2):", cards.nth(2).get_by_role("heading").text_content())
    print("last:", cards.last.get_by_role("heading").text_content())

    any_button = page.get_by_role("button", name="Add to cart").or_(page.get_by_role("button", name="Out of
stock"))
    print("or:", any_button.count())
    disabled_button = page.get_by_role("button").and_(page.locator("[disabled]"))
    print("and:", disabled_button.text_content())
```

Run it: `pytest -v -s`

Lesson: Filtering and chaining

### Strict mode: one action, one element

tests/test_strict.py

```
import pytest
from playwright.sync_api import Page, Error, expect


def test_strict_mode_refuses_to_guess(page: Page):
    page.goto("/products")
    add = page.get_by_role("button", name="Add to cart")
    expect(add).to_have_count(4)              # counting many is fine

    with pytest.raises(Error) as info:        # acting on many is not
        add.click()
    print("\n".join(info.value.message.split("\n")[:4]))
```

Run it: `pytest -v -s`

Lesson: Strict mode

### Working on one row of the orders table

| | |
|---|---|
| `page.goto("http://localhost:4173/orders")` | Open the orders page. It shows 5 of 7 orders per page. |
| `page.get_by_role("button", name="Total").click()` | Click the Total column header to sort by total, smallest first. |
| `page.get_by_role("row").filter(has_text="John").get_by_role("button", name="Delete").click()` | Find the row that mentions John, then the Delete button inside that row only. |
| `expect(page.get_by_role("status")).to_have_text("Order #1004 deleted")` | The status message confirms which order went away. |

| | |
|---|---|
| `page.get_by_role("button", name="Next").click()` | Go to the next page of the table. |
| `expect(page.get_by_text("Page 2 of 2")).to_be_visible()` | Check the page counter by its text. |

Lesson: Lists and tables

### Read every row of a table, then act on one row

tests/test_table.py

```
from playwright.sync_api import Page, expect


def test_read_a_table_and_act_on_one_row(page: Page):
    page.goto("/orders")
    rows = page.get_by_role("row").filter(has=page.get_by_role("cell"))   # skip the header row
    expect(rows).to_have_count(5)

    for row in rows.all():
        print(row.get_by_role("cell").all_text_contents())

    meera = rows.filter(has_text="Meera")
    meera.get_by_role("button", name="Delete").click()
    expect(page.get_by_role("status")).to_have_text("Order #1003 deleted")
    expect(rows.filter(has_text="Meera")).to_have_count(0)
```

Run it: `pytest -v -s`

Lesson: Lists and tables

## TABLES

### Table: html · role · example locator

| HTML | Role | Example locator |
|---|---|---|
| <button>, <input type=submit> | button | getByRole('button', { name: 'Sign in' }) |
| <a href> | link | getByRole('link', { name: 'Orders' }) |
| <h1> to <h6> | heading | getByRole('heading', { level: 1 }) |
| <input type=text>, <textarea> | textbox | getByRole('textbox', { name: 'Email' }) |
| <input type=search> | searchbox | getByRole('searchbox') |
| <select> | combobox | getByRole('combobox', { name: 'Sort by' }) |
| <input type=checkbox> and radio | checkbox, radio | getByRole('checkbox', { checked: true }) |
| <li>, <tr>, <td> | listitem, row, cell | getByRole('row').filter(...) |

Lesson: getByRole: the first choice

### Table: locator · finds · good for

| Locator | Finds | Good for |
|---|---|---|
| getByLabel | a form box by its <label> or aria-label | every form |
| getByPlaceholder | a box by its hint text | boxes with no label |
| getByText | the smallest element that contains the text | messages, paragraphs, labels |
| getByAltText | an image by its alt text | logos, product pictures |
| getByTitle | an element by its title attribute | icons, iframes |
| getByTestId | an element by data-testid | text that changes or is translated |

Lesson: Labels, placeholders, text, alt text, titles and test ids

**Table: you want · better choice · fragile choice**

| You want | Better choice | Fragile choice |
|---|---|---|
| the Sign in button | getByRole('button', { name: 'Sign in' }) | locator('#login > button.btn-primary') |
| the email box | getByLabel('Email') | locator('//form/div[1]/input') |
| the price of the mug | getByRole('listitem').filter({ hasText: 'Coffee mug' }).getByText(/₹/) | locator('li:nth-child(1) span:nth-child(3)') |

Lesson: CSS and XPath

**Table: method · keeps**

| Method | Keeps |
|---|---|
| filter(has_text="x") | elements that contain the text somewhere inside |
| has_not_text="x" | elements that do not contain the text |
| has=locator | elements that contain an element the locator finds |
| has_not=locator | elements that do not contain it |
| visible=True | only visible elements |
| first, last, nth(i) | one element by position, nth counts from 0 |
| a.and_(b), a.or_(b) | elements both match, or either matches |

Lesson: Filtering and chaining

### TIPS AND TRAPS

IN REAL TEAMS If getByRole cannot find a button that you can see, the page is probably missing a role or a name, and a screen reader user cannot use it either. That is a real accessibility bug worth reporting.

Lesson: getByRole: the first choice

IN REAL TEAMS Many companies use another attribute for test ids, such as data-qa or data-test. Tell Playwright once: `playwright.selectors.set_test_id_attribute("data-qa")`, or in pytest a fixture that calls it. After that, getByTestId reads the new attribute.

Lesson: Labels, placeholders, text, alt text, titles and test ids

COMMON MISTAKE Long XPaths copied from the browser tools, such as /html/body/div[2]/div/form/input, break as soon as one box is added to the page. Recorded tests from older tools are full of them, and they are the first thing to replace when a team moves to Playwright.

Lesson: CSS and XPath

COMMON MISTAKE first(), last() and nth() depend on the order of the page. When the order changes, they quietly pick another element. Prefer a filter by text when you can.

Lesson: Filtering and chaining

## 3.2 Actions and auto-waiting

An interactive auto-waiting lab, a real click on a covered, disabled and moving button, the actionability table from the docs, the call log of a timeout, force and trial, click options and hover, fill versus pressSequentially, a recorded walk through a feedback form, every form control, drag and drop by dragTo and by hand, and waits that replace sleeps.

### KEY IDEAS

- **Auto-waiting** Checks before every action. No sleeps needed.
- **Timeouts** The error says which check failed. Read it before using force.
- **Typing** fill for most boxes. pressSequentially for key by key.
- **Forms** check, selectOption, dragTo. Found by their labels.

- **Remember** Never sleep. Every action waits until the element is ready, and when you need more, wait for the exact thing you need.

CODE

### Click a button that is covered, disabled and moving

tests/test_auto_wait.py

```python
from playwright.sync_api import Page, expect


def test_actions_wait_until_the_element_is_ready(page: Page):
    page.goto("/widgets")
    # A loading cover hides the page for 0.8 s, and Pay now is disabled for 1 s.
    page.get_by_role("button", name="Pay now").click()
    expect(page.get_by_text("Payment done")).to_be_visible()

    # The offer appears 1.2 s after the click. The check waits for it.
    page.get_by_role("button", name="Load offers").click()
    expect(page.get_by_test_id("offer")).to_have_text("Buy 2 mugs, get 1 free")

    # This button slides in when the page opens. click waits until it stops moving.
    page.get_by_role("button", name="Catch me").click()
    expect(page.get_by_text("Caught it")).to_be_visible()
```

Run it: `pytest -v`

Lesson: Auto-waiting: no more sleeps

### A click that can never happen, and its call log

tests/test_call_log.py

```python
import pytest
from playwright.sync_api import Page, TimeoutError


def test_when_it_never_gets_ready_the_error_says_why(page: Page):
    page.goto("/products")
    sold_out = page.get_by_role("button", name="Out of stock")

    with pytest.raises(TimeoutError) as info:
        sold_out.click(timeout=2000)
    print(type(info.value).__name__)
    print("\n".join(info.value.message.split("\n")[:7]))
```

Run it: `pytest -v -s`

Lesson: When waiting is not enough

### Hover and focus

tests/test_hover.py

```python
from playwright.sync_api import Page, expect


def test_hover_and_focus_show_the_tooltip(page: Page):
    page.goto("/forms")
    help_icon = page.get_by_text("❓ Help")
    tooltip = page.get_by_role("tooltip")
    expect(tooltip).to_be_hidden()

    help_icon.hover()                       # the mouse rests on it
    expect(tooltip).to_have_text("We reply within two working days")

    page.mouse.move(0, 0)                   # move the mouse away
    expect(tooltip).to_be_hidden()

    help_icon.focus()                       # keyboard users reach it with Tab
    expect(tooltip).to_be_visible()
    print("tooltip:", tooltip.text_content())
```

Run it: `pytest -v -s`

Lesson: Clicking, hovering and focusing

### fill, pressSequentially and keyboard shortcuts

tests/test_keys.py

```python
from playwright.sync_api import Page, expect


def test_fill_type_key_by_key_and_press_shortcuts(page: Page):
    page.goto("/products")
    search = page.get_by_placeholder("Search products")
    expect(page.get_by_role("listitem")).to_have_count(5)

    search.fill("desk")                                  # sets the whole value at once
    print("after fill:", search.input_value(), "|", page.get_by_role("listitem").count(), "card")

    search.clear()
    search.press_sequentially("mug", delay=50)           # one key at a time, like a person
    print("after typing:", search.input_value())

    search.press("Control+A")
    search.press("Backspace")
    print("after select all and delete:", len(search.input_value()), "characters")

    page.goto("/forms")
    page.keyboard.press("Control+K")                     # a shortcut for the whole page
    expect(page.get_by_text("Search opened")).to_be_visible()
```

Run it: `pytest -v -s`

Lesson: Typing and keys

### Filling the feedback form

| | |
|---|---|
| `page.goto("http://localhost:4173/forms")` | Open the feedback form. |
| `page.get_by_label("Your name").fill("Asha")` | Type the name into the box labelled Your name. |
| `page.get_by_label("Topic").select_option("delivery")` | Choose Delivery in the Topic drop down. |
| `page.get_by_label("Happy").check()` | Pick the Happy radio button. |
| `page.get_by_label("Message").fill("The parcel came a day early.")` | Write a short message. |
| `page.get_by_label("Send me the newsletter").check()` | Tick the newsletter box. |
| `page.get_by_role("button", name="Send feedback").click()` | Send the form. Playwright scrolls the button into view by itself. |
| `expect(page.get_by_role("status")).to_contain_text("Topic: delivery, mood: happy")` | The thank you line shows every value the page received. |

Lesson: Checkboxes, radio buttons, drop downs and dates

### Text boxes, drop downs, radio buttons, checkboxes, dates and sliders

tests/test_form.py

```python
from playwright.sync_api import Page, expect


def test_fill_every_kind_of_form_control(page: Page):
    page.goto("/forms")
    page.get_by_label("Your name").fill("Asha")
    page.get_by_label("Topic").select_option("delivery")            # by value
    page.get_by_label("Topic").select_option(label="Payment")       # or by the text you see
    page.get_by_label("Happy").check()                              # a radio button
    page.get_by_label("Order date").fill("2026-10-01")              # dates use yyyy-mm-dd
    page.get_by_label("Message").fill("The parcel came a day early.")
    page.get_by_label("Send me the newsletter").set_checked(True)
    page.get_by_label("Volume").fill("8")                           # a slider

    print("topic:", page.get_by_label("Topic").input_value())
    print("happy checked:", page.get_by_label("Happy").is_checked())
    print("date:", page.get_by_label("Order date").input_value())
    print("volume:", page.get_by_label("Volume").input_value())

    page.get_by_role("button", name="Send feedback").click()
```

```
    expect(page.get_by_role("status")).to_have_text(
        "Thanks Asha! Topic: payment, mood: happy, files: no files, newsletter: yes")
```

Run it: `pytest -v -s`

Lesson: Checkboxes, radio buttons, drop downs and dates

### Drag and drop with dragTo, and by hand with the mouse

tests/test_drag.py

```
from playwright.sync_api import Page, expect


def test_drag_tasks_to_the_done_lane(page: Page):
    page.goto("/forms")
    todo = page.get_by_role("list", name="To do")
    done = page.get_by_role("list", name="Done")

    todo.get_by_text("Pack the order").drag_to(done)
    expect(done.get_by_role("listitem")).to_have_text(["Pack the order"])

    # The same by hand with the mouse, for widgets that need finer control.
    todo.get_by_text("Print the label").hover()
    page.mouse.down()
    done.hover()
    done.hover()            # a second move, so the page gets its dragover event
    page.mouse.up()

    expect(done.get_by_role("listitem")).to_have_count(2)
    print("Done lane:", done.get_by_role("listitem").all_text_contents())
    print("To do lane:", todo.get_by_role("listitem").count(), "tasks")
```

Run it: `pytest -v -s`

Lesson: Drag and drop and the mouse

### Wait for a response, an element or a condition

tests/test_wait.py

```
from playwright.sync_api import Page, expect


def test_wait_for_what_you_need_never_for_a_fixed_time(page: Page):
    # Start waiting for the API answer before the page asks for it.
    with page.expect_response(lambda r: "/api/products" in r.url) as answer:
        page.goto("/products")
    print("API answered with status", answer.value.status)

    page.get_by_text("Loading products…").wait_for(state="detached")
    print("cards now:", page.get_by_role("listitem").count())

    page.goto("/widgets")
    page.wait_for_function("() => !document.querySelector('#cover')")
    print("loading cover left:", page.locator("#cover").count())
    expect(page.get_by_role("button", name="Pay now")).to_be_enabled()
```

Run it: `pytest -v -s`

Lesson: Waiting without sleeping

TABLES

### Table: action · visible · stable · receives events

| Action | Visible | Stable | Receives events | Enabled | Editable |
|---|---|---|---|---|---|
| click, dblclick, check, uncheck, setChecked, tap | ✓ | ✓ | ✓ | ✓ | |
| hover, dragTo | ✓ | ✓ | ✓ | | |
| screenshot | ✓ | ✓ | | | |
| fill, clear | ✓ | | | ✓ | ✓ |
| selectOption | ✓ | | | ✓ | |
| selectText | ✓ | | | | |
| scrollIntoViewIfNeeded | | ✓ | | | |
| focus, blur, press, pressSequentially, setInputFiles, dispatchEvent | | | | | |

Lesson: Auto-waiting: no more sleeps

### Table: option · what it does · when to use it

| Option | What it does | When to use it |
|---|---|---|
| force=True | skips the checks that are not essential, such as receives events | a custom widget that hides its real input under a styled label |
| trial=True | runs every check but does not click | to wait until something could be clicked, without clicking it |
| timeout=5000 | a different timeout for this one action | one action that is known to be slow |

Lesson: When waiting is not enough

### Table: you want · code

| You want | Code |
|---|---|
| a double click | dblclick() |
| a right click | click(button="right") |
| Shift and click | click(modifiers=["Shift"]) |
| Control on Windows and Linux, Command on macOS | modifiers=["ControlOrMeta"] |
| a click at the top left corner | position={"x": 0, "y": 0} |
| the mouse resting on it | hover() |
| a tap on a touch screen | tap() with has_touch=True on the context |

Lesson: Clicking, hovering and focusing

### Table: method · sends · use for

| Method | Sends | Use for |
|---|---|---|
| fill(text) | the whole value at once | almost every text box |
| pressSequentially(text) | one key event per character | auto complete, masks that format as you type |
| press(key) | one key or a combination | Enter, Tab, Escape, ArrowDown, Control+A |
| keyboard.press(key) | a key to whatever has focus | page shortcuts such as Control+K |
| clear() | an empty value | emptying a box |

Lesson: Typing and keys

**Table: control · method · read it back**

| Control | Method | Read it back |
|---|---|---|
| text box, text area | fill | inputValue |
| checkbox, radio button | check, uncheck, setChecked | isChecked or toBeChecked |
| drop down (select) | selectOption by value, label or index, or a list for multi select | inputValue or toHaveValue |
| date, time, color, range | fill with the standard format, such as 2026-10-01 | inputValue |
| file input | setInputFiles | the files list |

Lesson: Checkboxes, radio buttons, drop downs and dates

**Table: wait for · method**

| Wait for | Method |
|---|---|
| an element to appear, hide or go away | locator.wait_for(state=...) |
| a new URL | page.wait_for_url |
| a network response | with page.expect_response(...) |
| anything the page can compute | page.wait_for_function |
| a popup, download or dialog | with page.expect_popup() and friends |

Lesson: Waiting without sleeping

### TIPS AND TRAPS

COMMON MISTAKE Reaching for force is often a sign the test found the wrong element, or that a real user could not click it either. Read the call log first.

Lesson: When waiting is not enough

NEW IN PLAYWRIGHT Old tutorials use type(). It was renamed pressSequentially in Playwright 1.38 and the old name is deprecated. If you see it in a code base, it does the same thing.

Lesson: Typing and keys

IN REAL TEAMS Many teams add a lint rule or a code review check that rejects waitForTimeout. A suite with 200 tests and a 2 second sleep in each spends almost 7 minutes doing nothing on every run.

Lesson: Waiting without sleeping

## 3.3 Assertions that wait for the page

How a web-first assertion retries, a plain read against a retrying check, fifteen checks on one page, text and list checks for sorting, negative checks, a real failure report, soft assertions in all four languages, plain value checks on API data, expect.poll and toPass, and a recorded walk through a search.

### KEY IDEAS

- **Web-first** Retries until true. 5 seconds by default.
- **Text** Exact or contains. Lists check order too.
- **Negative** Waits for things to go. toHaveCount(0) for none.
- **Soft** Keep going. Report every problem.
- **Remember** Check the page with assertions that retry, and never read a value first and check it later.

### CODE

**A web-first check against a plain read**

`tests/test_retry.py`

```
from playwright.sync_api import Page, expect
```

```python
def test_a_web_first_check_waits_a_plain_read_does_not(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Load offers").click()
    offer = page.get_by_test_id("offer")      # shows up 1.2 s after the click

    print("is_visible() right now:", offer.is_visible())   # a snapshot, no waiting
    expect(offer).to_be_visible()                          # retries until it shows
    print("is_visible() after the check:", offer.is_visible())
```

Run it: `pytest -v -s`

Lesson: Web-first assertions retry

### Fifteen checks on the sign in page

tests/test_many_checks.py

```python
import re
from playwright.sync_api import Page, expect


def test_one_check_for_each_thing_a_user_can_notice(page: Page):
    page.goto("/login")
    expect(page).to_have_title("Sign in · Brainy Shop")
    expect(page).to_have_url(re.compile(r"/login$"))

    email = page.get_by_label("Email")
    expect(email).to_be_empty()
    expect(email).to_be_editable()
    expect(email).to_have_attribute("type", "email")
    expect(email).to_have_role("textbox")
    email.fill("asha@shop.example")
    expect(email).to_have_value("asha@shop.example")
    expect(email).to_be_focused()

    expect(page.get_by_role("checkbox", name="Remember me")).not_to_be_checked()
    expect(page.get_by_role("alert")).to_be_hidden()
    sign_in = page.get_by_role("button", name="Sign in")
    expect(sign_in).to_be_enabled()
    expect(sign_in).to_have_css("background-color", "rgb(22, 163, 74)")
    expect(sign_in).to_be_in_viewport()
    expect(page.get_by_role("form")).to_have_accessible_name("Sign in")
    print("all 15 checks passed")
```

Run it: `pytest -v -s`

Lesson: The assertions you will use every day

### toHaveText and toContainText for one element and for lists

tests/test_text.py

```python
import re
from playwright.sync_api import Page, expect


def test_text_checks_for_one_element_and_for_a_list(page: Page):
    page.goto("/products")
    names = page.get_by_role("listitem").get_by_role("heading")

    # A list: every element, in this order.
    expect(names).to_have_text(["Coffee mug", "Desk lamp", "Gel pen", "Notebook", "Water bottle"])
    # A list: some elements contain these texts, in this order.
    expect(names).to_contain_text(["mug", "pen"])

    expect(page.get_by_role("heading", level=1)).to_have_text("products", ignore_case=True)
    expect(page.get_by_test_id("product-3")).to_contain_text("₹1,299")
    expect(page.get_by_test_id("product-3")).to_have_text(re.compile(r"Desk lamp.*Home"))

    page.get_by_role("combobox", name="Sort by").select_option("low")
    expect(names).to_have_text(["Gel pen", "Notebook", "Coffee mug", "Water bottle", "Desk lamp"])
    print("sorted:", names.all_text_contents())
```

Run it: `pytest -v -s`

Lesson: Checking text and lists

## Checking a search, one assertion at a time

| | |
|---|---|
| `page.goto("http://localhost:4173/products")` | Open the products page. |
| `expect(page.get_by_role("listitem")).to_have_count(5)` | The check waits until the slow product list has loaded. |
| `page.get_by_placeholder("Search products").fill("bottle")` | Search for bottle. |
| `expect(page.get_by_role("listitem")).to_contain_text("Water bottle")` | One card is left, and it is the right one. |
| `expect(page.get_by_placeholder("Search products")).to_have_value("bottle")` | The search box keeps what was typed. |
| `page.get_by_placeholder("Search products").fill("phone")` | Search for something the shop does not sell. |
| `expect(page.get_by_role("button", name="Add to cart")).to_have_count(0)` | No Add to cart buttons are left. A negative check also waits. |

Lesson: Checking that something is not there

## Checks that something is gone

tests/test_negative.py

```python
from playwright.sync_api import Page, expect


def test_negative_checks_wait_too(page: Page):
    page.goto("/products")
    expect(page.get_by_text("Loading products…")).not_to_be_visible()   # waits until it goes away
    expect(page.get_by_text("No products match your search")).to_have_count(0)

    page.get_by_placeholder("Search products").fill("phone")
    expect(page.get_by_role("listitem")).to_have_text("No products match your search")
    expect(page.get_by_role("button", name="Add to cart")).to_have_count(0)
    expect(page.get_by_role("button", name="Add to cart")).not_to_be_attached()
    print("nothing to buy when searching for phone")
```

Run it: `pytest -v -s`

Lesson: Checking that something is not there

## What a failing check looks like

tests/test_failing.py

```python
from playwright.sync_api import Page, expect


def test_a_failing_check_explains_itself(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)
    expect(page.get_by_test_id("product-2"), "the mug should cost 149 in the sale") \
        .to_contain_text("₹149", timeout=2000)
```

Run it: `pytest -v`

Lesson: Reading a failure

## Soft checks: keep going, then fail with every problem

tests/test_soft.py

```python
from playwright.sync_api import Page, expect


def test_soft_checks_report_every_problem_at_once(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)

    expect.soft(page.get_by_test_id("product-1")).to_contain_text("₹25", timeout=1000)          # wrong on purpose
    expect.soft(page.get_by_test_id("product-4")).to_contain_text("Add to cart", timeout=1000)  # wrong on purpose
    expect.soft(page.get_by_test_id("product-5")).to_contain_text("₹349")                       # right
    print("the test kept going after the failed checks")
```

Run it: `pytest -v -s`

Lesson: Soft assertions

### Plain checks on data you already have

tests/test_values.py

```python
from playwright.sync_api import Page, expect


def test_plain_value_checks_for_data_you_already_have(page: Page):
    response = page.request.get("/api/products?delay=0")
    expect(response).to_be_ok()                      # a Playwright check for API answers
    products = response.json()

    assert len(products) == 5                        # plain Python checks
    assert "Desk lamp" in [p["name"] for p in products]
    assert products[0] == {"id": 1, "name": "Gel pen", "price": 20, "stock": 120, "category": "Stationery"}
    assert products[2]["price"] > 1000
    print("cheapest:", min(p["price"] for p in products))
```

Run it: `pytest -v -s`

Lesson: Plain values, polling and blocks

## TABLES

### Table: checks · python

| Checks | Python |
|---|---|
| visible or hidden | to_be_visible, to_be_hidden |
| text | to_have_text, to_contain_text |
| how many | to_have_count |
| box value | to_have_value, to_have_values, to_be_empty |
| state | to_be_checked, to_be_enabled, ... |
| attributes and style | to_have_attribute, to_have_class, ... |
| accessibility | to_have_role, ... |
| the page | to_have_title, to_have_url |
| an API answer | to_be_ok |
| pictures and structure | to_match_aria_snapshot |

Lesson: The assertions you will use every day

### Table: assertion · with a string · with a list of strings

| Assertion | With a string | With a list of strings |
|---|---|---|
| toHaveText | the whole text must match | exactly these elements, in this order |
| toContainText | the text must contain it | some elements, in this order, containing these parts |

Lesson: Checking text and lists

## TIPS AND TRAPS

COMMON MISTAKE A rule that catches many flaky tests: never put await inside expect. Write `expect(locator).to_be_visible()`, not `assert locator.is_visible()`.

Lesson: Web-first assertions retry

REMEMBER Change the timeout of one check with its timeout option, or of all checks: `expect.set_options(timeout=10_000)` in conftest.py. Raise it for a slow test environment, not to hide a slow page.

Lesson: Reading a failure

IN REAL TEAMS Soft checks are handy for a page with many independent fields, such as an invoice or a profile. Do not use them for steps the rest of the test depends on, such as signing in.

Lesson: Soft assertions

## STEP 4 The test runner

### 4.1 The test runner: organise, tag, parameterise and configure

Which runner each language uses, what happens in a test run, groups, tags, skip and expected failures in all four languages, running only smoke tests, parameterised search tests, and one test run as a desktop and as a phone from configuration.

KEY IDEAS

- **Runner** Finds and runs tests. One per language.
- **Tags** Run a slice. Smoke now, regression tonight.
- **Data** One test, many rows. Each row is its own result.
- **Config** Settings in one place. Projects for each device.
- **Remember** Tests say what to check. The runner and its configuration decide where, how and with what settings they run.

CODE

**Group, tag, skip and mark known failures**

tests/test_products_page.py

```python
import pytest
from playwright.sync_api import Page, expect


class TestProductsPage:
    @pytest.mark.smoke
    def test_shows_five_products(self, page: Page):
        page.goto("/products")
        expect(page.get_by_role("listitem")).to_have_count(5)

    @pytest.mark.regression
    def test_sorts_by_price(self, page: Page):
        page.goto("/products")
        page.get_by_label("Sort by").select_option("low")
        expect(page.get_by_role("listitem").first).to_contain_text("Gel pen")

    @pytest.mark.skip(reason="the wish list is not built yet")
    def test_adds_a_product_to_the_wish_list(self, page: Page):
        pass

    @pytest.mark.regression
    @pytest.mark.xfail(reason="SHOP-42: phones arrive next month", strict=True)
    def test_sells_phones(self, page: Page):
        page.goto("/products")
        page.get_by_placeholder("Search products").fill("phone")
        expect(page.get_by_role("button", name="Add to cart")).to_have_count(1, timeout=1000)
```

Run it: `pytest -v`

Lesson: Grouping, tags and annotations

**Run only the tests with one tag**

tests/test_products_page.py

```python
import pytest
from playwright.sync_api import Page, expect


class TestProductsPage:
    @pytest.mark.smoke
```

```python
    def test_shows_five_products(self, page: Page):
        page.goto("/products")
        expect(page.get_by_role("listitem")).to_have_count(5)

    @pytest.mark.regression
    def test_sorts_by_price(self, page: Page):
        page.goto("/products")
        page.get_by_label("Sort by").select_option("low")
        expect(page.get_by_role("listitem").first).to_contain_text("Gel pen")

    @pytest.mark.skip(reason="the wish list is not built yet")
    def test_adds_a_product_to_the_wish_list(self, page: Page):
        pass

    @pytest.mark.regression
    @pytest.mark.xfail(reason="SHOP-42: phones arrive next month", strict=True)
    def test_sells_phones(self, page: Page):
        page.goto("/products")
        page.get_by_placeholder("Search products").fill("phone")
        expect(page.get_by_role("button", name="Add to cart")).to_have_count(1, timeout=1000)
```

Run it: `pytest -v -m smoke`

Lesson: Running a slice of the suite

### One test, many rows of data

tests/test_search.py

```python
import pytest
from playwright.sync_api import Page, expect


@pytest.mark.parametrize("term, count, first", [
    ("mug", 1, "Coffee mug"),
    ("o", 3, "Coffee mug"),
    ("pen", 1, "Gel pen"),
    ("BOTTLE", 1, "Water bottle"),
])
def test_search(page: Page, term: str, count: int, first: str):
    page.goto("/products")
    page.get_by_placeholder("Search products").fill(term)
    cards = page.get_by_role("listitem")
    expect(cards).to_have_count(count)
    expect(cards.first).to_contain_text(first)
```

Run it: `pytest -v`

Lesson: Parameterised tests

### Configuration that every test receives

pytest.ini

```ini
[pytest]
addopts =
    --base-url http://localhost:4173
    --browser chromium
    --screenshot only-on-failure
    --tracing retain-on-failure
```

conftest.py

```python
import pytest


# Options for every browser context that the page fixture opens.
@pytest.fixture(scope="session")
def browser_context_args(browser_context_args):
    return {**browser_context_args, "locale": "en-IN", "timezone_id": "Asia/Kolkata"}
```

tests/test_config.py

```python
from playwright.sync_api import Page


def test_the_config_reaches_every_test(page: Page):
    page.goto("/about-you")
```

```
    print("| viewport", page.viewport_size["width"],
          "| language", page.locator("#lang").text_content(),
          "| time zone", page.locator("#tz").text_content())
```

Run it: `pytest -s -q pytest -s -q --device "Pixel 7"`

Lesson: Configuration in one place

## TABLES

### Table: language · test runner · how playwright plugs in

| Language | Test runner | How Playwright plugs in |
|---|---|---|
| Python | pytest | the pytest-playwright plugin gives page, context and browser fixtures and CLI options |

Lesson: One library, a runner per language

### Table: you want · python (pytest)

| You want | Python (pytest) |
|---|---|
| a group | a class TestSomething |
| a tag | @pytest.mark.smoke |
| skip | @pytest.mark.skip(reason=...) |
| skip on a condition | @pytest.mark.skipif(condition, reason=...) |
| known to fail | @pytest.mark.xfail(strict=True) |
| broken, fix later | @pytest.mark.skip with a reason |
| give it more time | a timeout marker from a plugin |
| run only this one | -k name on the command line |

Lesson: Grouping, tags and annotations

### Table: setting · pytest-playwright

| Setting | pytest-playwright |
|---|---|
| base URL | --base-url |
| browser | --browser firefox |
| headed | --headed |
| device | --device "Pixel 7" |
| locale and other context options | browser_context_args fixture |
| test timeout | a plugin such as pytest-timeout |
| screenshots, video, trace | --screenshot, --video, --tracing |

Lesson: Configuration in one place

## TIPS AND TRAPS

COMMON MISTAKE A test.only left in the code silently skips every other test. Set forbidOnly in the CI config so such a run fails instead.

Lesson: Grouping, tags and annotations

IN REAL TEAMS In real projects the rows often come from a CSV or JSON file, for example 50 postcodes for a delivery check. Read the file at the top of the test file and loop over it, or use @CsvFileSource in JUnit and a list built from the file in pytest.

Lesson: Parameterised tests

## 4.2 Fixtures, hooks and set up once

The order hooks run in, the built in fixtures of each language, the life of a fixture, a signed in page fixture with a clean up, an automatic fixture that fails tests on JavaScript errors, and set up that checks the shop and creates data once per run.

### KEY IDEAS

- **Hooks** Around every test. Simple and visible.
- **Fixtures** Only when asked for. Set up, use, tear down.
- **Automatic** Every test gets it. Guards and recordings.
- **Once per run** Health check, data. Before any test.
- **Remember** Put set up where it belongs: in the test if one test needs it, in a fixture if several do, and in a setup step if the whole run does.

### CODE

#### Before and after hooks, and the order they run in

tests/test_hooks.py

```python
import pytest
from playwright.sync_api import Page, expect


@pytest.fixture(scope="module", autouse=True)
def once_for_this_file():
    print("\nbefore all: once for this file")
    yield
    print("\nafter all: once at the end")


@pytest.fixture(autouse=True)
def open_products(page: Page, request):
    print(f"\nbefore each: open the products page for {request.node.originalname}")
    page.goto("/products")
    yield
    print(f"\nafter each: {request.node.originalname} done")


def test_shows_five_products(page: Page):
    expect(page.get_by_role("listitem")).to_have_count(5)


def test_finds_the_lamp(page: Page):
    page.get_by_placeholder("Search products").fill("lamp")
    expect(page.get_by_role("listitem")).to_have_count(1)
```

Run it: `pytest -s -q`

Lesson: Before and after hooks

#### A signed in page for every test that asks for it

conftest.py

```python
import pytest
from playwright.sync_api import Page, expect


@pytest.fixture
def signed_in_page(page: Page):
    # Set up: sign in as Asha.
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()
    print("\nfixture set up: signed in")

    yield page                                  # the test runs here

    # Tear down: leave the cart empty for the next test.
```

```
    page.request.delete("/api/cart")
    print("\nfixture tear down: cart emptied")
```

tests/test_cart.py

```
from playwright.sync_api import Page, expect


def test_add_a_mug_to_the_cart(signed_in_page: Page):
    page = signed_in_page
    page.get_by_test_id("product-2").get_by_role("button", name="Add to cart").click()
    expect(page.get_by_role("status")).to_have_text("Coffee mug added to your cart")
    expect(page.get_by_test_id("cart-count")).to_have_text("1")
    print("test: the mug is in the cart")
```

Run it: `pytest -s -q`

Lesson: Fixtures: set up on demand

### An automatic fixture that fails a test when the page throws an error

conftest.py

```
import pytest
from playwright.sync_api import Page


@pytest.fixture(autouse=True)       # every test gets it, without asking for it
def fail_on_page_errors(page: Page):
    errors = []
    page.on("pageerror", lambda error: errors.append(error.message))
    yield
    assert errors == [], f"the page threw errors: {errors}"
```

tests/test_pages.py

```
from playwright.sync_api import Page, expect


def test_the_products_page_has_no_errors(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)


def test_the_widgets_page_breaks(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Break the page").click()
    expect(page.get_by_role("heading", name="Console and errors")).to_be_visible()
    # the test itself passes, the fixture finds the error afterwards
```

Run it: `pytest -v`

Lesson: Automatic fixtures

### Set up once: check the shop and create test data

conftest.py

```
import pytest
from playwright.sync_api import Playwright, expect


@pytest.fixture(scope="session", autouse=True)      # once for the whole run
def shop_data(playwright: Playwright, base_url: str):
    api = playwright.request.new_context(base_url=base_url)
    expect(api.get("/api/health")).to_be_ok()
    response = api.post("/api/people", data={"name": "Test Customer", "email": "test@shop.example"})
    assert response.status == 201
    print("\nsetup: created person", response.json()["id"])
    yield
    api.dispose()
```

tests/test_people.py

```
from playwright.sync_api import Page


def test_the_test_data_is_there(page: Page):
    people = page.request.get("/api/people").json()
```

```
    print("test: people in the shop:", [p["name"] for p in people])
    assert len(people) == 1
```

Run it: `pytest -s -q`

Lesson: Set up once for the whole run

### TABLES

**Table: when · pytest**

| When | pytest |
|---|---|
| before each test | a fixture, code before yield |
| after each test | a fixture, code after yield |
| once before all | a fixture with scope="module" or "session" |
| once after all | the same fixture, after yield |

Lesson: Before and after hooks

**Table: fixture · pytest-playwright**

| Fixture | pytest-playwright |
|---|---|
| a fresh page | page |
| its browser context | context |
| the shared browser | browser |
| an API client | page.request, or playwright.request.new_context() |
| which browser | browser_name |
| Playwright itself | playwright |

Lesson: Fixtures: set up on demand

**Table: scope · pytest**

| Scope | pytest |
|---|---|
| each test | scope="function" (the default) |
| each worker process | not built in (pytest-xdist workers use session) |
| the file or class | scope="module" or "class" |
| the whole run | scope="session" |

Lesson: Set up once for the whole run

### TIPS AND TRAPS

IN REAL TEAMS Hooks run for every test in the file. Fixtures run only for tests that ask for them, and they can depend on other fixtures. As a suite grows, teams move set up from hooks into fixtures.

Lesson: Fixtures: set up on demand

## 4.3 Parallel runs, retries, timeouts and sharding

Four tests on two workers in each language, two tests kept apart with a lock, a flaky test that passes on retry, the five kinds of timeout with one that is too short, and a suite split into two shards.

### KEY IDEAS

- **Workers** Tests at the same time. Each with its own browser.
- **Isolation** No shared data. Lock what must be shared.
- **Retries** Try again on failure. Flaky is still a bug.
- **Shards** Split across machines. Merge the reports.

- **Remember** Parallel runs only work when every test can run alone, in any order, next to any other test.

CODE

### Four tests, two at a time

tests/test_products.py

```
from playwright.sync_api import Page, expect


def test_shows_five_products(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)


def test_finds_the_lamp(page: Page):
    page.goto("/products")
    page.get_by_placeholder("Search products").fill("lamp")
    expect(page.get_by_role("listitem")).to_have_count(1)
```

tests/test_orders.py

```
from playwright.sync_api import Page, expect


def test_page_1_shows_five_orders(page: Page):
    page.goto("/orders")
    expect(page.get_by_role("row")).to_have_count(6)        # a header row and five orders


def test_page_2_is_the_last_page(page: Page):
    page.goto("/orders")
    page.get_by_role("button", name="Next").click()
    expect(page.get_by_text("Page 2 of 2")).to_be_visible()
```

Run it: `pytest -n 2 -v`

Lesson: Workers: tests at the same time

### Two tests that must not run at the same time

tests/test_people.py

```
import pytest
from playwright.sync_api import Page, expect


# Both tests count the people in the shop. The same group keeps them on one worker, one after the other.
@pytest.mark.xdist_group("people")
def test_add_meera(page: Page):
    before = len(page.request.get("/api/people").json())
    page.request.post("/api/people", data={"name": "Meera", "email": "meera@shop.example"})
    assert len(page.request.get("/api/people").json()) == before + 1


@pytest.mark.xdist_group("people")
def test_add_chen(page: Page):
    before = len(page.request.get("/api/people").json())
    page.request.post("/api/people", data={"name": "Chen", "email": "chen@shop.example"})
    assert len(page.request.get("/api/people").json()) == before + 1


def test_shows_five_products(page: Page):          # no group, runs on any worker
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)
```

Run it: `pytest -n 3 --dist loadgroup -v`

Lesson: Parallel tests must not share data

### A flaky test that passes on the second try

tests/test_offer.py

```
from playwright.sync_api import Page, expect
```

```python
def test_the_offer_shows_up(page: Page, request):
    attempt = request.node.execution_count          # set by pytest-rerunfailures
    print(f"\nattempt {attempt}")
    page.goto("/widgets")
    page.get_by_role("button", name="Load offers").click()
    # Flaky on purpose: the first try waits only 0.5 s, but the offer needs 1.2 s.
    wait = 500 if attempt == 1 else 5_000
    expect(page.get_by_test_id("offer")).to_be_visible(timeout=wait)
```

Run it: `pytest -v -s --reruns 2`

Lesson: Retries and flaky tests

### Timeouts that are long enough, and one that is too short

conftest.py

```python
import pytest
from playwright.sync_api import Page, expect

expect.set_options(timeout=5_000)                   # each expect check


@pytest.fixture(autouse=True)
def timeouts(page: Page):
    page.set_default_timeout(10_000)                # each click, fill, check and so on
    page.set_default_navigation_timeout(15_000)     # each goto
```

tests/test_timeouts.py

```python
from playwright.sync_api import Page, expect


def test_waits_long_enough_for_the_offer(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Load offers").click()
    expect(page.get_by_test_id("offer")).to_have_text("Buy 2 mugs, get 1 free")   # the 5 s default is
enough


def test_gives_up_too_early(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Load offers").click()
    expect(page.get_by_test_id("offer")).to_be_visible(timeout=1_000)            # 1 s, but the offer needs
1.2 s


def test_a_click_with_its_own_timeout(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Pay now").click(timeout=3_000)              # this click only
    expect(page.get_by_text("Payment done")).to_be_visible()
```

Run it: `pytest -v`

Lesson: Timeouts

### Split the suite in two, as two CI machines would

tests/test_products.py

```python
from playwright.sync_api import Page, expect


def test_shows_five_products(page: Page):
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)


def test_finds_the_lamp(page: Page):
    page.goto("/products")
    page.get_by_placeholder("Search products").fill("lamp")
    expect(page.get_by_role("listitem")).to_have_count(1)
```

tests/test_orders.py

```python
from playwright.sync_api import Page, expect
```

```python
def test_page_1_shows_five_orders(page: Page):
    page.goto("/orders")
    expect(page.get_by_role("row")).to_have_count(6)        # a header row and five orders


def test_page_2_is_the_last_page(page: Page):
    page.goto("/orders")
    page.get_by_role("button", name="Next").click()
    expect(page.get_by_text("Page 2 of 2")).to_be_visible()
```

Run it: `pytest --splits 2 --group 1 -v && pytest --splits 2 --group 2 -v`

Lesson: Sharding across machines

## TABLES

**Table: pytest**

| | pytest |
|---|---|
| how | the pytest-xdist plugin, worker processes |
| turn it on | pytest -n 4, or -n auto |
| how many | -n 4 |
| tests inside one file | spread over workers |
| turn it off | leave out -n |

Lesson: Workers: tests at the same time

**Table: pytest**

| | pytest |
|---|---|
| set retries | the pytest-rerunfailures plugin, --reruns 2 |
| which try is this | request.node.execution_count, starts at 1 |
| how it is reported | rerun |
| fail the run on flaky tests | not built in |

Lesson: Retries and flaky tests

**Table: limit · default · python**

| Limit | Default | Python |
|---|---|---|
| one test | 30 s | not built in (the pytest-timeout plugin adds it) |
| one expect check | 5 s | expect.set_options(timeout=5_000) |
| one action | no limit in Playwright Test, 30 s in the libraries | page.set_default_timeout() |
| one navigation | no limit in Playwright Test, 30 s in the libraries | page.set_default_navigation_timeout() |
| the whole run | no limit | not built in |

Lesson: Timeouts

## TIPS AND TRAPS

COMMON MISTAKE The Playwright docs advise against it: isolated tests are faster and easier to retry.

Lesson: Parallel tests must not share data

REMEMBER When a check times out, the fix is almost never a bigger number. Ask what the page was waiting for. A longer timeout is right only when the app really is that slow, such as a report that takes 20 s to build.

Lesson: Timeouts

IN REAL TEAMS Shards are balanced by test count, not by time. With fullyParallel: true, Playwright Test splits single tests, so the shards come out even. Without it, whole files are split, so keep files a similar size. pytest-split can balance by recorded durations.

Lesson: Sharding across machines

## STEP 5 Network and APIs

### 5.1 Network: watch, mock and change requests

Logging API traffic and reading one response, mocking the products API, a 500 and a broken connection, selling out the lamp in a real answer, recording and replaying a HAR file, and mocking the live stock WebSocket.

KEY IDEAS

- **Watch** See every call. Wait for the one you need.
- **Mock** Answer with your data. No server needed.
- **Change** Real answer, one field changed. The rest stays real.
- **Replay** Record once. Same data every run.
- **Remember** Mock what you do not own or cannot control. Keep a few tests against the real API, so the mocks do not drift away from it.

CODE

**Listen to the traffic and wait for one response**

tests/test_watch.py

```
from urllib.parse import urlparse
from playwright.sync_api import Page, Request, Response, expect


def show_request(request: Request):
    if "/api/" in request.url:
        print(">>", request.method, urlparse(request.url).path)


def show_response(response: Response):
    if "/api/" in response.url:
        print("<<", response.status, urlparse(response.url).path)


def test_the_products_come_from_the_api(page: Page):
    page.on("request", show_request)
    page.on("response", show_response)

    with page.expect_response("**/api/products") as products_call:   # wait while the page loads
        page.goto("/products")
    response = products_call.value

    assert response.ok
    products = response.json()
    print(f"the API sent {len(products)} products")
    expect(page.get_by_role("listitem")).to_have_count(len(products))
```

Run it: `pytest -s -q`

Lesson: Watch the traffic

**Answer the products API with your own data**

tests/test_mock.py

```
from playwright.sync_api import Page, Route, expect

PRODUCTS = [
    {"id": 7, "name": "Tea kettle", "price": 899, "stock": 4, "category": "Kitchen"},
    {"id": 8, "name": "Pencil box", "price": 150, "stock": 0, "category": "Stationery"},
]
```

```python
def test_shows_the_products_the_api_sends(page: Page):
    page.route("**/api/products", lambda route: route.fulfill(json=PRODUCTS))

    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(2)
    expect(page.get_by_test_id("product-7")).to_contain_text("₹899")
    expect(page.get_by_test_id("product-8").get_by_role("button")).to_have_text("Out of stock")
```

Run it: `pytest -v`

Lesson: Mock an API

### Make the API fail, and check the page copes

tests/test_errors.py

```python
from playwright.sync_api import Page, expect

SORRY = "Sorry, we could not load the products. Please try again."


def test_a_server_error_shows_a_friendly_message(page: Page):
    page.route("**/api/products", lambda route: route.fulfill(status=500, json={"error": "Database is
down"}))
    page.goto("/products")
    expect(page.get_by_role("alert")).to_have_text(SORRY)


def test_a_broken_connection_shows_the_same_message(page: Page):
    page.route("**/api/products", lambda route: route.abort())
    page.goto("/products")
    expect(page.get_by_role("alert")).to_have_text(SORRY)
```

Run it: `pytest -v`

Lesson: Errors that are hard to cause

### Keep the real answer, change one field

tests/test_sold_out.py

```python
from playwright.sync_api import Page, Route, expect


def sell_out_the_lamp(route: Route):
    response = route.fetch()                        # ask the real server
    products = response.json()
    for product in products:                        # then change one field
        if product["name"] == "Desk lamp":
            product["stock"] = 0
    route.fulfill(response=response, json=products)


def test_the_lamp_shows_as_sold_out(page: Page):
    page.route("**/api/products", sell_out_the_lamp)

    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)   # the other products are real
    expect(page.get_by_test_id("product-3").get_by_role("button")).to_have_text("Out of stock")
```

Run it: `pytest -v`

Lesson: Change a real response

### Record the API into a HAR file, then replay it

tests/test_har.py

```python
from playwright.sync_api import Page, expect


def test_record_the_products_api(page: Page):
    page.route_from_har("hars/products.har", url="**/api/products", update=True)
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)
```

```python
def test_replay_it_from_the_file(page: Page):
    page.route_from_har("hars/products.har", url="**/api/products")   # no update: answers come from the file
    page.goto("/products")
    expect(page.get_by_role("listitem")).to_have_count(5)
```

Run it: `pytest -v && ls hars`

Lesson: Record and replay with HAR

### Mock a WebSocket feed

tests/test_live_stock.py

```python
import json
from playwright.sync_api import Page, WebSocketRoute, expect


def test_shows_the_live_stock_from_the_server(page: Page):
    page.goto("/widgets")
    expect(page.get_by_test_id("live-stock")).to_have_text("Desk lamp: 8 left")


def answer(ws: WebSocketRoute, message):
    if message == "subscribe":
        ws.send(json.dumps({"product": "Desk lamp", "left": 0}))


def test_a_mocked_feed_says_the_lamp_sold_out(page: Page):
    # no real server is contacted
    page.route_web_socket("**/ws/stock", lambda ws: ws.on_message(lambda message: answer(ws, message)))
    page.goto("/widgets")
    expect(page.get_by_test_id("live-stock")).to_have_text("Desk lamp: sold out")
```

Run it: `pytest -v`

Lesson: WebSockets

## TABLES

### Table: to · python

| To | Python |
|---|---|
| log every request | page.on("request", ...) |
| log every response | page.on("response", ...) |
| wait for one response | with page.expect_response(...) |
| wait for one request | with page.expect_request(...) |
| see failed requests | page.on("requestfailed", ...) |

Lesson: Watch the traffic

### Table: step · what you write

| Step | What you write |
|---|---|
| record | routeFromHAR(file, { url, update: true }) and run the test once |
| edit | change the JSON in the body file next to the HAR, if you want different data |
| replay | routeFromHAR(file, { url }) without update |
| refresh | run once more with update: true when the API changes |

Lesson: Record and replay with HAR

## TIPS AND TRAPS

REMEMBER

The pattern in page.route is a glob: ** matches any part of the address, including slashes, and * matches any part between two slashes. A regular expression works too. Routes added with context.route catch requests from every page of the context, including new tabs.

Lesson: Mock an API

IN REAL TEAMS Routes can also make tests faster. page.route('**/*.{png,jpg,jpeg}', route => route.abort()) stops images from loading in tests that do not need them.

Lesson: Errors that are hard to cause

REMEMBER route.continue can change the request on its way out instead: a different header, method or body. For example, route.continue({ headers: { ...route.request().headers(), "x-test": "yes" } }).

Lesson: Change a real response

COMMON MISTAKE Replay matches the address and the method exactly, and for POST also the body. A request that matches nothing in the file is aborted, unless you pass notFound: fallback.

Lesson: Record and replay with HAR

## 5.2 API testing and signing in once

A full create, read, update and delete test of the people API with a validation error, a token sent in a header, a cart filled through the API and checked in the page, and a saved sign in reused by every test.

### KEY IDEAS

- **API tests** No browser at all. Status, data, errors.
- **Tokens** Proof of who calls. Keep them in secrets.
- **Mix** API for set up. Page for the check.
- **Sign in once** Save the state. Every test starts signed in.
- **Remember** Use the page for what the user sees. Use the API for everything you only need to get ready.

### CODE

**Create, read, update and delete through the API**

conftest.py

```
import pytest
from playwright.sync_api import APIRequestContext, Playwright


@pytest.fixture
def api(playwright: Playwright, base_url: str) -> APIRequestContext:
    context = playwright.request.new_context(base_url=base_url)
    yield context
    context.dispose()
```

tests/test_people_api.py

```
from playwright.sync_api import APIRequestContext, expect


def test_create_read_update_and_delete_a_person(api: APIRequestContext):
    created = api.post("/api/people", data={"name": "Kiran", "email": "kiran@shop.example"})
    assert created.status == 201
    kiran = created.json()
    assert created.headers["location"] == f"/api/people/{kiran['id']}"

    read = api.get(f"/api/people/{kiran['id']}")
    expect(read).to_be_ok()
    assert read.json() == {"id": kiran["id"], "name": "Kiran", "email": "kiran@shop.example", "city": None}

    updated = api.patch(f"/api/people/{kiran['id']}", data={"city": "Pune"})
    assert updated.json()["city"] == "Pune"

    assert api.delete(f"/api/people/{kiran['id']}").status == 204
    assert api.get(f"/api/people/{kiran['id']}").status == 404


def test_a_bad_email_is_refused(api: APIRequestContext):
    response = api.post("/api/people", data={"name": "Nobody", "email": "not-an-email"})
```

```
    assert response.status == 422
    assert response.json() == {"error": "name and a valid email are required"}
```

Run it: `pytest -v`

Lesson: API tests without a browser

### Sign in through the API and send the token

tests/test_token.py

```
from playwright.sync_api import APIRequestContext, Playwright


def test_the_token_opens_the_cart(api: APIRequestContext, playwright: Playwright, base_url: str):
    assert api.get("/api/cart").status == 401          # nobody is signed in yet

    login = api.post("/api/login", data={"email": "asha@shop.example", "password": "s3cret"})
    token, name = login.json()["token"], login.json()["name"]
    print(f"\nsigned in as {name}")

    # A fresh client that sends the token with every call, and has no cookies.
    asha = playwright.request.new_context(base_url=base_url, extra_http_headers={"Authorization": f"Bearer
{token}"})
    asha.post("/api/cart", data={"productId": 2, "qty": 2})
    cart = asha.get("/api/cart").json()
    print(cart)
    assert cart["count"] == 2
    assert cart["total"] == 398
    asha.dispose()
```

Run it: `pytest -s -q`

Lesson: Tokens and headers

### Arrange through the API, check through the page

tests/test_cart_setup.py

```
from playwright.sync_api import Page, expect


def test_a_lamp_added_through_the_api_shows_in_the_cart_page(page: Page):
    # Sign in through the API. page.request shares cookies with the page.
    login = page.request.post("/api/login", data={"email": "asha@shop.example", "password": "s3cret"})
    token = login.json()["token"]
    page.add_init_script(f"localStorage.setItem('token', '{token}')")     # the shop keeps its token here

    page.request.post("/api/cart", data={"productId": 3})                  # arrange the cart without
clicking

    page.goto("/cart")
    expect(page.get_by_text("Hello, Asha")).to_be_visible()
    expect(page.get_by_role("row", name="Desk lamp")).to_contain_text("₹1,299")
    expect(page.get_by_test_id("cart-count")).to_have_text("1")
```

Run it: `pytest -v`

Lesson: API for set up, page for checks

### Sign in once, save the state, start every test signed in

conftest.py

```
from pathlib import Path
import pytest
from playwright.sync_api import Browser, expect

STATE = Path("playwright/.auth/asha.json")


@pytest.fixture(scope="session")
def browser_context_args(browser_context_args, browser: Browser, base_url: str):
    # Sign in once for the whole run, and save cookies and local storage.
    context = browser.new_context(base_url=base_url)
    page = context.new_page()
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
```

```
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_text("Hello, Asha")).to_be_visible()     # wait until signing in has finished
    STATE.parent.mkdir(parents=True, exist_ok=True)            # Python does not create the folder for you
    context.storage_state(path=STATE)
    context.close()
    return {**browser_context_args, "storage_state": STATE}    # every test starts signed in
```

tests/test_signed_in.py

```
import re
from playwright.sync_api import Page, expect


def test_the_products_page_greets_asha(page: Page):
    page.goto("/products")                              # no sign in steps here
    expect(page.get_by_text("Hello, Asha")).to_be_visible()


def test_the_cart_is_open_to_her(page: Page):
    page.goto("/cart")
    expect(page.get_by_role("heading", name="Your cart")).to_be_visible()
    expect(page).to_have_url(re.compile(r"/cart$"))  # not sent to /login
```

Run it: `pytest -v`

Lesson: Sign in once for the whole suite

## TABLES

**Table: python**

| | Python |
|---|---|
| the client | playwright.request.new_context() |
| send JSON | api.post(url, data=...) |
| check status 200 to 299 | expect(response).to_be_ok() |
| read JSON | response.json() |

Lesson: API tests without a browser

**Table: need · approach**

| Need | Approach |
|---|---|
| all tests can share one account | one setup step and one state file, as above |
| tests change data of the account | one account per worker, named with the worker index |
| two roles, such as customer and admin | two state files, asha.json and ravi.json, and a project or fixture for each |
| the app keeps its session in sessionStorage | storageState does not save sessionStorage, so save it yourself and restore it with addInitScript |

Lesson: Sign in once for the whole suite

## TIPS AND TRAPS

COMMON MISTAKE Keep real passwords and tokens out of test code. Read them from environment variables, such as process.env.SHOP_PASSWORD, os.environ["SHOP_PASSWORD"] or System.getenv("SHOP_PASSWORD"), and set them as secrets in CI. The framework chapter shows a .env file for local runs.

Lesson: Tokens and headers

REMEMBER page.request (and context.request) share cookies with the browser context. A sign in through page.request signs in the page too, for apps that use cookies. The shop also keeps a token in local storage, which is why the example adds it with addInitScript.

Lesson: API for set up, page for checks

COMMON MISTAKE The state file holds a live session that anyone could use. Add playwright/.auth to .gitignore, and let each run create it fresh.

Lesson: Sign in once for the whole suite

## STEP 6 Debugging and quality

### 6.1 Debugging failed tests: errors, traces, reports and the Inspector

A failed check read line by line in four languages, a failed click that leaves a screenshot, a video and a trace, a trace with named steps, verbose logs, and real screenshots of the trace viewer, the HTML report, UI mode and the Inspector.

KEY IDEAS

- **Error** Expected and received. Read it first.
- **Evidence** Screenshot, video, trace. Kept on failure.
- **Trace** Every action replayed. DOM, network, console.
- **Inspector** Run it yourself. One step at a time.
- **Remember** Read the error, then the trace, and only then run the test by hand. Most failures are solved at the first step.

CODE

**A failed check, and what the runner prints**

tests/test_greeting.py

```
from playwright.sync_api import Page, expect


def test_greets_asha_after_sign_in(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.locator("#hello")).to_have_text("Hello, Asha!")   # the page says Hello, Asha
```

Run it: `pytest -v`

Lesson: Read the error first

**A screenshot, a video and a trace when a test fails**

pytest.ini

```
[pytest]
addopts = --base-url http://localhost:4173 --screenshot only-on-failure --video retain-on-failure --tracing
retain-on-failure
```

tests/test_cart.py

```
import pytest
from playwright.sync_api import Page, expect


@pytest.fixture(autouse=True)
def signed_in(page: Page):
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.get_by_role("listitem")).to_have_count(5)


def test_add_the_lamp_to_the_cart(page: Page):
    page.get_by_test_id("product-3").get_by_role("button", name="Add to cart").click()
    expect(page.get_by_test_id("cart-count")).to_have_text("1")


def test_add_the_notebook_to_the_cart(page: Page):
```

```
    # the Notebook is out of stock, so this button never appears
    page.get_by_test_id("product-4").get_by_role("button", name="Add to cart").click(timeout=3000)
```

Run it: `pytest -v ; find test-results -type f | sort`

Lesson: Screenshots, videos and traces on failure

### A trace with named steps

tests/test_mug.py

```
from playwright.sync_api import BrowserContext, Page, expect


def test_buy_a_mug(context: BrowserContext, page: Page):
    context.tracing.start(screenshots=True, snapshots=True, sources=True)

    context.tracing.group("sign in")
    page.goto("/login")
    page.get_by_label("Email").fill("asha@shop.example")
    page.get_by_label("Password").fill("s3cret")
    page.get_by_role("button", name="Sign in").click()
    expect(page.locator("#hello")).to_have_text("Hello, Asha")
    context.tracing.group_end()

    context.tracing.group("add the mug")
    page.get_by_test_id("product-2").get_by_role("button", name="Add to cart").click()
    expect(page.get_by_test_id("cart-count")).to_have_text("1")
    context.tracing.group_end()

    context.tracing.stop(path="traces/buy-a-mug.zip")
```

Run it: `pytest -v && ls traces`

Lesson: The trace viewer

### Every Playwright call, written to the terminal

tests/test_lamp.py

```
from playwright.sync_api import Page, expect


def test_find_the_lamp(page: Page):
    page.goto("/products")
    page.get_by_placeholder("Search products").fill("lamp")
    expect(page.get_by_role("listitem")).to_have_count(1)
```

Run it: `DEBUG=pw:api pytest -s`

Lesson: Verbose logs

## TABLES

### Table: part · what it tells you

| Part | What it tells you |
|---|---|
| the locator | which element the check or action looked for |
| expected and received | what the test wanted and what the page had, side by side |
| the timeout | how long Playwright kept trying before it gave up |
| the call log | each try: did the locator find an element, was it visible, enabled and stable, what did it contain |

Lesson: Read the error first

### Table: the error says · it usually means

| The error says | It usually means |
|---|---|
| received a different text or count | the page changed, or the expected value in the test is out of date |
| element(s) not found, or waiting for locator | the locator is wrong, a step before it did not happen, or the page did not load |
| strict mode violation, resolved to 2 elements | the locator is too loose, so narrow it with a role, a name or a parent |
| element is not enabled, or is not visible | the app has not finished something, such as loading data or validating a form |
| Test timeout of 30000ms exceeded | one call waited for something that never happened, and the call log names it |

Lesson: Read the error first

### Table: setting · pytest-playwright

| Setting | pytest-playwright |
|---|---|
| screenshot | --screenshot only-on-failure |
| video | --video retain-on-failure |
| trace | --tracing retain-on-failure |
| where | test-results/ |
| other values | on, off |

Lesson: Screenshots, videos and traces on failure

### Table: open a trace · command

| Open a trace | Command |
|---|---|
| Python | playwright show-trace test-results/.../trace.zip |
| any language, no install | drag the file onto trace.playwright.dev, which reads it in your browser |

Lesson: The trace viewer

### Table: python

| | Python |
|---|---|
| open the Inspector | PWDEBUG=1 pytest -s |
| one test only | PWDEBUG=1 pytest -s -k notebook |
| stop at a line | page.pause() |
| see the source | shown by itself |
| watch slowly | --headed --slowmo 500 |

Lesson: Step through a test with the Inspector

### Table: variable · what it prints

| Variable | What it prints |
|---|---|
| DEBUG=pw:api | every Playwright call and the steps inside it |
| DEBUG=pw:browser | what the browser process prints, useful when it will not start |
| DEBUG=pw:protocol | every message between Playwright and the browser, very long |
| PWDEBUG=console | a playwright object in the browser DevTools, to try locators in the console |

Lesson: Verbose logs

## TIPS AND TRAPS

REMEMBER Read the received value carefully. A missing full stop, an extra space or a different currency sign is the most common cause of a red check, and the error shows it to you directly.

Lesson: Read the error first

IN REAL TEAMS Teams often use trace: 'on-first-retry' in CI. A first run that fails is retried with tracing on, so the trace is recorded only for tests that failed at least once.

Lesson: Screenshots, videos and traces on failure

REMEMBER Inside the trace viewer, click an action and then use the Before and After tabs to compare the page. The Pick locator button lets you try a new locator on the saved page.

Lesson: The trace viewer

COMMON MISTAKE UI mode, the HTML report and the VS Code extension are part of Playwright Test, so they are there for JavaScript and TypeScript. Python teams usually add pytest-html, and Java teams add Allure or ExtentReports. The reporting chapter builds each of them.

Lesson: The HTML report and UI mode

COMMON MISTAKE Remove page.pause() before you commit. It needs a headed browser and is only useful while you debug.

Lesson: Step through a test with the Inspector

REMEMBER Browser console messages and page errors are a different stream. Collect them with page.on("console") and page.on("pageerror"), as the fixtures chapter does, or read them in the Console tab of the trace.

Lesson: Verbose logs

## 6.2 Visual, accessibility and time checks

A screenshot comparison that catches a style bug with a real diff picture, aria snapshots of the menu and a product card, an axe scan that finds a missing alt text and low contrast, and a sale date and an idle sign out tested with a fake clock.

### KEY IDEAS

- **Screenshot** How it looks. Baseline and diff.
- **Aria snapshot** How it reads. Every language.
- **axe scan** Who can use it. WCAG rules.
- **Fake clock** Any date, any delay. No waiting.
- **Remember** Screenshots check pixels, aria snapshots check structure, axe checks access, and the clock checks time. Use each for the mistakes it is good at.

### CODE

**Check the structure of the page with an aria snapshot**

tests/test_structure.py

```
from playwright.sync_api import Page, expect


def test_the_main_menu_has_four_links(page: Page):
    page.goto("/products")
    expect(page.get_by_role("navigation", name="Main")).to_match_aria_snapshot("""
        - navigation "Main":
          - link "Products"
          - link "Feedback"
          - link "Widgets"
          - link "Orders"
    """)


def test_the_notebook_card_says_it_is_out_of_stock(page: Page):
    page.goto("/products")
    print(page.get_by_test_id("product-4").aria_snapshot())   # print it once, then paste it below
    expect(page.get_by_test_id("product-4")).to_match_aria_snapshot("""
        - listitem:
          - heading "Notebook" [level=3]
          - text: Stationery ₹60
```

```
          - button "Out of stock" [disabled]
    """)
```

Run it: `pytest -v -s`

Lesson: Aria snapshots

### Scan a page for accessibility problems with axe

tests/test_offers.py

```python
from axe_playwright_python.sync_playwright import Axe
from playwright.sync_api import Page


def test_the_offers_page_has_no_accessibility_problems(page: Page):
    page.goto("/a11y")
    results = Axe().run(page, options={"runOnly": ["wcag2a", "wcag2aa"]})   # the WCAG 2 A and AA rules
    for v in results.response["violations"]:
        print(f"{v['id']} ({v['impact']}): {len(v['nodes'])} element(s). {v['help']}")
    assert [v["id"] for v in results.response["violations"]] == []
```

Run it: `pytest -v -s`

Lesson: Accessibility scans with axe

### Control dates and timers with a fake clock

tests/test_clock.py

```python
from playwright.sync_api import Page, expect


def test_the_sale_is_not_live_the_day_before(page: Page):
    page.clock.set_fixed_time("2026-12-24T12:00:00Z")
    page.goto("/clock")
    expect(page.get_by_role("status")).to_have_text("The Christmas sale is coming soon")


def test_the_sale_goes_live_on_christmas_day(page: Page):
    page.clock.set_fixed_time("2026-12-25T12:00:00Z")
    page.goto("/clock")
    expect(page.get_by_role("status")).to_have_text("The Christmas sale is live")


def test_an_idle_user_is_signed_out_after_5_minutes(page: Page):
    page.clock.install(time="2026-12-24T10:00:00Z")
    page.goto("/clock")
    page.clock.run_for("05:00")                          # 5 minutes of timers, in a moment
    expect(page.get_by_role("alert")).to_have_text("You were signed out")
```

Run it: `pytest -v`

Lesson: Fake time with page.clock

## TABLES

### Table: option · what it does

| Option | What it does |
|---|---|
| --update-snapshots | saves new baselines instead of comparing, after a change you meant to make |
| maxDiffPixels, maxDiffPixelRatio | how many pixels, or what share of them, may differ |
| threshold | how different one pixel may be before it counts, from 0 to 1, default 0.2 |
| mask: [locator] | covers parts that change every run, such as a date or an advert, with a box |
| animations: "disabled" | stops CSS animations so the picture is the same every time (the default) |
| stylePath | adds a style sheet only while taking the picture, for example to hide a clock |

Lesson: Visual comparisons

**Table: python**

| | Python |
|---|---|
| print a snapshot | locator.aria_snapshot() |
| check a snapshot | to_match_aria_snapshot("""...""") |
| keep it in a file | not built in |

Lesson: Aria snapshots

**Table: what axe found · why it matters · the fix**

| What axe found | Why it matters | The fix |
|---|---|---|
| image-alt: the lamp image has no alt text | a screen reader cannot say what the picture shows | add alt="Desk lamp" |
| color-contrast: light blue text on a light background | people with low vision cannot read it | use a darker colour, with a contrast ratio of at least 4.5 to 1 |

Lesson: Accessibility scans with axe

**Table: package · language · made by**

| Package | Language | Made by |
|---|---|---|
| @axe-core/playwright | JavaScript and TypeScript | Deque |
| com.deque.html.axe-core:playwright | Java | Deque |
| axe-playwright-python | Python | community |

Lesson: Accessibility scans with axe

**Table: method · what it does**

| Method | What it does |
|---|---|
| setFixedTime(time) | Date.now() always returns this time. Timers still run normally. |
| install({ time }) | replaces Date and every timer. Call it before the page loads. |
| runFor("05:00") | moves forward, firing every timer that falls due on the way |
| fastForward("05:00") | jumps forward and fires due timers at most once, like a laptop waking from sleep |
| pauseAt(time), resume() | stops time at a moment to check the page, then lets it run again |
| setSystemTime(time) | changes the date without firing timers, like a user changing the system clock |

Lesson: Fake time with page.clock

## TIPS AND TRAPS

IN REAL TEAMS Baselines depend on the operating system and the browser, which is why the file name ends in chromium-linux. Make them on the same system that runs the tests, usually the CI image or the Playwright Docker image, and commit them to the repository.

Lesson: Visual comparisons

COMMON MISTAKE toHaveScreenshot is part of Playwright Test. Python and Java have page.screenshot() to save a picture, but no comparison built in. Teams add a library for it or use a visual testing service. The aria snapshot below works in every language.

Lesson: Visual comparisons

REMEMBER Codegen can record aria snapshots too: use the Assert snapshot button in the toolbar while recording, and the snapshot of the element you click is added to the test.

Lesson: Aria snapshots

COMMON MISTAKE An automatic scan finds only part of the problems. It cannot tell whether alt text is useful or whether the tab order makes sense. Keep a few manual checks with a keyboard and a screen reader, and use the scan to stop the easy mistakes from coming back.

Lesson: Accessibility scans with axe

IN REAL TEAMS An old page may have many known problems. Scan with .exclude() for the parts you will fix later, or compare the list of violations with a saved list, so new problems still fail the test.

Lesson: Accessibility scans with axe

REMEMBER Use setFixedTime when the page only reads the date, and install with runFor when it uses setTimeout or setInterval. Either way the test runs in milliseconds and gives the same result on every day of the year.

Lesson: Fake time with page.clock

## STEP 7 Framework design

### 7.1 Page objects and components

A LoginPage, a ProductsPage built from Header and ProductCard components, and fixtures that hand out signed in page objects, each run for real in four languages.

KEY IDEAS

- **Page object** One class per screen. Locators in one place.
- **Component** Parts that repeat. Root locator first.
- **Fixtures** Hand them out. No set up in tests.
- **Checks** Stay in the test. Easy to read.
- **Remember** The test says what, the page object says how. A change to the screen is fixed in one class.

CODE

**Step 1: one page object for the sign in page**

pages/login_page.py

```
from playwright.sync_api import Page


# Everything a test needs to know about the sign in page, in one place
class LoginPage:
    def __init__(self, page: Page):
        self.page = page
        self.email = page.get_by_label("Email")
        self.password = page.get_by_label("Password")
        self.sign_in_button = page.get_by_role("button", name="Sign in")
        self.error = page.get_by_role("alert")

    def goto(self):
        self.page.goto("/login")

    def sign_in(self, email: str, password: str):
        self.email.fill(email)
        self.password.fill(password)
        self.sign_in_button.click()
```

tests/test_login.py

```
from playwright.sync_api import Page, expect

from pages.login_page import LoginPage


def test_asha_can_sign_in(page: Page):
    login_page = LoginPage(page)
```

```
    login_page.goto()
    login_page.sign_in("asha@shop.example", "s3cret")
    expect(page.get_by_text("Hello, Asha")).to_be_visible()


def test_a_wrong_password_is_refused(page: Page):
    login_page = LoginPage(page)
    login_page.goto()
    login_page.sign_in("asha@shop.example", "wrong")
    expect(login_page.error).to_have_text("Wrong email or password")
```

pytest.ini

```
[pytest]
addopts = --base-url http://localhost:4173
pythonpath = .
```

Run it: `pytest -v`

Lesson: Why page objects

### Step 2: components for parts that repeat

components/product_card.py

```
from playwright.sync_api import Locator


# One product card. Every locator starts from the card, so it never finds another product's button.
class ProductCard:
    def __init__(self, root: Locator):
        self.root = root
        self.price = root.locator(".price")
        self.button = root.get_by_role("button")

    def add_to_cart(self):
        self.root.get_by_role("button", name="Add to cart").click()
```

components/header.py

```
from playwright.sync_api import Page


# The header is on every page, so it is a component, not a page
class Header:
    def __init__(self, page: Page):
        root = page.get_by_role("banner")
        self.greeting = root.locator("#hello")
        self.cart_count = root.get_by_test_id("cart-count")
        self.sign_out = root.get_by_role("link", name="Sign out")
```

pages/products_page.py

```
from playwright.sync_api import Page

from components.header import Header
from components.product_card import ProductCard


class ProductsPage:
    def __init__(self, page: Page):
        self.page = page
        self.header = Header(page)
        self.search = page.get_by_placeholder("Search products")
        self.cards = page.get_by_role("listitem")

    def goto(self):
        self.page.goto("/products")

    def card(self, name: str) -> ProductCard:
        return ProductCard(self.cards.filter(has=self.page.get_by_role("heading", name=name)))
```

tests/test_cart.py

```
from playwright.sync_api import Page, expect

from pages.login_page import LoginPage
from pages.products_page import ProductsPage
```

```python
def test_asha_adds_the_lamp_and_the_mug(page: Page):
    login_page = LoginPage(page)
    login_page.goto()
    login_page.sign_in("asha@shop.example", "s3cret")

    products = ProductsPage(page)
    expect(products.header.greeting).to_have_text("Hello, Asha")
    products.card("Desk lamp").add_to_cart()
    expect(products.header.cart_count).to_have_text("1")
    products.card("Coffee mug").add_to_cart()
    expect(products.header.cart_count).to_have_text("2")


def test_the_notebook_is_out_of_stock(page: Page):
    products = ProductsPage(page)
    products.goto()
    notebook = products.card("Notebook")
    expect(notebook.price).to_have_text("₹60")
    expect(notebook.button).to_have_text("Out of stock")
    expect(notebook.button).to_be_disabled()
```

Run it: `pytest -v`

Lesson: Components for repeated parts

### Step 3: page objects as fixtures

conftest.py

```python
import pytest
from playwright.sync_api import Page

from pages.login_page import LoginPage
from pages.products_page import ProductsPage


# Tests ask for login_page or products_page by name, like they ask for page
@pytest.fixture
def login_page(page: Page) -> LoginPage:
    return LoginPage(page)


@pytest.fixture
def products_page(page: Page, login_page: LoginPage) -> ProductsPage:
    login_page.goto()
    login_page.sign_in("asha@shop.example", "s3cret")   # products_page always starts signed in
    return ProductsPage(page)
```

tests/test_mug.py

```python
from playwright.sync_api import expect

from pages.products_page import ProductsPage


def test_the_mug_costs_199_rupees_and_can_be_added(products_page: ProductsPage):
    mug = products_page.card("Coffee mug")
    expect(mug.price).to_have_text("₹199")
    mug.add_to_cart()
    expect(products_page.header.cart_count).to_have_text("1")


def test_the_search_finds_one_lamp(products_page: ProductsPage):
    products_page.search.fill("lamp")
    expect(products_page.cards).to_have_count(1)
```

Run it: `pytest -v`

Lesson: Handing out page objects

### TABLES

**Table: without page objects · with page objects**

| Without page objects | With page objects |
|---|---|
| locators copied into every test | each locator written once |
| a UI change breaks many files | a UI change is fixed in one class |
| tests read as clicks and fills | tests read as steps: signIn, addToCart |
| new team members copy old tests | new tests reuse a small, named set of methods |

Lesson: Why page objects

**Table: pytest**

| | pytest |
|---|---|
| where page objects are created | @pytest.fixture in conftest.py |
| how a test gets one | by name: products_page |
| only built when used | yes |

Lesson: Handing out page objects

**Table: rule · why**

| Rule | Why |
|---|---|
| one class per screen or component | easy to find, small enough to read |
| locators by role, label or test id | they survive style changes, as the locators chapter showed |
| methods are user actions: signIn, addToCart | tests read like steps, not clicks |
| checks live in tests | the test shows what it proves |
| no sleeps and no waits for timeouts | Playwright actions and checks already wait |
| no test data inside the class | pass it in, so one page object serves many tests |
| a method returns the next page object when the screen changes | signIn() can return ProductsPage, so the flow is clear |

Lesson: Rules teams follow

### TIPS AND TRAPS

REMEMBER Return locators, or small components, and leave the checks in the test. A test that says expect(notebook.button).toBeDisabled() shows what it proves. A page object method called checkNotebook() hides it.

Lesson: Components for repeated parts

IN REAL TEAMS Large suites sometimes add one more layer of business steps, such as checkout(user, items), built from page objects. Add it when you notice the same five page object calls in many tests, not before.

Lesson: Rules teams follow

## 7.2 Framework design, step by step

The same framework built and run in TypeScript, JavaScript, Python and Java: the runner finds the tests, the full suite passes, the smoke set runs alone, and every file is shown with why it is there.

### KEY IDEAS

- **Settings** From the environment. No secrets in Git.
- **Layers** One job each. Data, API, pages.
- **Fixtures** Fresh session per test. Safe in parallel.
- **Tests** By feature, tagged. Steps and checks only.

- **Remember** Framework code and tests live apart. Tests say what, the framework says how, and the environment says where.

CODE

### Step 1: the project, its settings and its runner config

pytest.ini

```
[pytest]
testpaths = tests
pythonpath = .
addopts = --browser chromium --tracing retain-on-failure --screenshot only-on-failure
markers =
    smoke: quick checks that run on every commit
    cart: the cart and checkout
```

framework/config.py

```
import os

from dotenv import load_dotenv

load_dotenv()   # reads .env when it exists, so a laptop and CI can use different settings

# Settings that change between machines (a laptop, CI, staging) live here and nowhere else
BASE_URL = os.getenv("BASE_URL", "http://localhost:4173")
```

Run it: `pytest --collect-only -q`

Lesson: Step 1: project, settings and runner config

### Step 2: data, API helpers, components, page objects and fixtures

framework/data/users.py

```
import os
from dataclasses import dataclass


@dataclass(frozen=True)
class User:
    email: str
    password: str
    name: str


# Test accounts. Real projects read passwords from CI secrets, never from the repository.
ASHA = User("asha@shop.example", os.getenv("ASHA_PASSWORD") or "s3cret", "Asha")
RAVI = User("ravi@shop.example", os.getenv("RAVI_PASSWORD") or "admin123", "Ravi")
```

framework/data/products.py

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Product:
    id: int
    name: str
    price: str


# The products the tests use, with the ids the API expects
GEL_PEN = Product(1, "Gel pen", "₹20")
COFFEE_MUG = Product(2, "Coffee mug", "₹199")
DESK_LAMP = Product(3, "Desk lamp", "₹1,299")
NOTEBOOK = Product(4, "Notebook", "₹60")
WATER_BOTTLE = Product(5, "Water bottle", "₹349")
```

framework/api/shop_api.py

```
from playwright.sync_api import APIRequestContext

from framework.data.users import User


# Fast set up through the API, so UI tests only click what they are testing
```

```python
class ShopApi:
    def __init__(self, request: APIRequestContext):
        self.request = request

    def sign_in(self, user: User) -> str:
        response = self.request.post("/api/login", data={"email": user.email, "password": user.password})
        assert response.ok, f"Sign in failed for {user.email}: {response.status}"
        return response.json()["token"]

    def add_to_cart(self, token: str, product_id: int, qty: int = 1):
        response = self.request.post("/api/cart", data={"productId": product_id, "qty": qty},
                                     headers={"Authorization": f"Bearer {token}"})
        assert response.ok, f"Could not add product {product_id}: {response.status}"
```

framework/components/header.py

```python
from playwright.sync_api import Page


# The header is on every page, so it is a component that every page object shares
class Header:
    def __init__(self, page: Page):
        root = page.get_by_role("banner")
        self.greeting = root.locator("#hello")
        self.cart_count = root.get_by_test_id("cart-count")
        self.sign_out = root.get_by_role("link", name="Sign out")
```

framework/components/product_card.py

```python
from playwright.sync_api import Locator


# One product card. Every locator starts from the card, so it never finds another product's button.
class ProductCard:
    def __init__(self, root: Locator):
        self.root = root
        self.price = root.locator(".price")
        self.button = root.get_by_role("button")

    def add_to_cart(self):
        self.root.get_by_role("button", name="Add to cart").click()
```

framework/pages/base_page.py

```python
from playwright.sync_api import Page

from framework.components.header import Header


# What every page shares: the browser page, its address and the header
class BasePage:
    path = "/"

    def __init__(self, page: Page):
        self.page = page
        self.header = Header(page)

    def open(self):
        self.page.goto(self.path)
```

framework/pages/login_page.py

```python
from playwright.sync_api import Page

from framework.data.users import User
from framework.pages.base_page import BasePage


class LoginPage(BasePage):
    path = "/login"

    def __init__(self, page: Page):
        super().__init__(page)
        self.email = page.get_by_label("Email")
        self.password = page.get_by_label("Password")
        self.sign_in_button = page.get_by_role("button", name="Sign in")
```

```
        self.error = page.get_by_role("alert")

    def sign_in(self, user: User):
        self.email.fill(user.email)
        self.password.fill(user.password)
        self.sign_in_button.click()
```

framework/pages/products_page.py

```
from playwright.sync_api import Page

from framework.components.product_card import ProductCard
from framework.pages.base_page import BasePage


class ProductsPage(BasePage):
    path = "/products"

    def __init__(self, page: Page):
        super().__init__(page)
        self.search = page.get_by_placeholder("Search products")
        self.cards = page.get_by_role("list", name="Products").get_by_role("listitem")
        self.no_results = page.get_by_text("No products match your search")

    def card(self, name: str) -> ProductCard:
        return ProductCard(self.cards.filter(has=self.page.get_by_role("heading", name=name)))

    def search_for(self, text: str):
        self.search.fill(text)
```

framework/pages/cart_page.py

```
from playwright.sync_api import Page

from framework.pages.base_page import BasePage


class CartPage(BasePage):
    path = "/cart"

    def __init__(self, page: Page):
        super().__init__(page)
        self.items = page.get_by_role("table", name="Cart items").locator("tbody tr")
        self.total = page.get_by_test_id("total")
        self.place_order_button = page.get_by_role("button", name="Place order")
        self.status = page.get_by_role("status")

    def place_order(self):
        self.page.once("dialog", lambda dialog: dialog.accept())   # the shop asks to confirm the order
        self.place_order_button.click()
```

conftest.py

```
from dataclasses import dataclass
from typing import Generator

import pytest
from playwright.sync_api import Page, Playwright

from framework import config
from framework.api.shop_api import ShopApi
from framework.data.users import ASHA, User
from framework.pages.cart_page import CartPage
from framework.pages.login_page import LoginPage
from framework.pages.products_page import ProductsPage


@dataclass
class Session:
    user: User
    token: str


@pytest.fixture(scope="session")
def base_url() -> str:
```

```
    return config.BASE_URL                      # pytest-playwright uses it for page.goto("/login")


@pytest.fixture
def api(playwright: Playwright, base_url: str) -> Generator[ShopApi, None, None]:
    request = playwright.request.new_context(base_url=base_url)
    yield ShopApi(request)
    request.dispose()


# A fresh session made through the API: fast, and no two tests share a cart
@pytest.fixture
def session(api: ShopApi, page: Page) -> Session:
    token = api.sign_in(ASHA)
    page.add_init_script(f"window.localStorage.setItem('token', '{token}')")
    return Session(ASHA, token)


@pytest.fixture
def login_page(page: Page) -> LoginPage:
    return LoginPage(page)


@pytest.fixture
def products_page(page: Page) -> ProductsPage:
    return ProductsPage(page)


@pytest.fixture
def cart_page(page: Page) -> CartPage:
    return CartPage(page)
```

Run it: `pytest -v`

Lesson: Step 2: data, API helpers, page objects and fixtures

### Step 3: tests by feature, tagged so a smaller set can run

tests/smoke/test_sign_in.py

```
from dataclasses import replace

import pytest
from playwright.sync_api import expect

from framework.data.users import ASHA
from framework.pages.login_page import LoginPage
from framework.pages.products_page import ProductsPage

pytestmark = pytest.mark.smoke


def test_asha_signs_in_with_the_right_password(login_page: LoginPage, products_page: ProductsPage):
    login_page.open()
    login_page.sign_in(ASHA)
    expect(products_page.header.greeting).to_have_text("Hello, Asha")


def test_a_wrong_password_shows_an_error(login_page: LoginPage):
    login_page.open()
    login_page.sign_in(replace(ASHA, password="wrong"))
    expect(login_page.error).to_have_text("Wrong email or password")
```

tests/products/test_search.py

```
from playwright.sync_api import expect

from framework.data.products import DESK_LAMP, NOTEBOOK
from framework.pages.products_page import ProductsPage


def test_finds_the_lamp_by_name(products_page: ProductsPage):
    products_page.open()
    products_page.search_for("lamp")
    expect(products_page.cards).to_have_count(1)
    expect(products_page.card(DESK_LAMP.name).price).to_have_text(DESK_LAMP.price)
```

```python

def test_says_so_when_nothing_matches(products_page: ProductsPage):
    products_page.open()
    products_page.search_for("piano")
    expect(products_page.no_results).to_be_visible()


def test_the_notebook_cannot_be_added(products_page: ProductsPage):
    products_page.open()
    expect(products_page.card(NOTEBOOK.name).button).to_be_disabled()
```

tests/cart/test_add_to_cart.py

```python
import pytest
from playwright.sync_api import expect

from framework.data.products import COFFEE_MUG, DESK_LAMP
from framework.pages.cart_page import CartPage
from framework.pages.products_page import ProductsPage

pytestmark = pytest.mark.cart


def test_adds_two_products_from_their_cards(session, products_page: ProductsPage, cart_page: CartPage):
    products_page.open()
    expect(products_page.header.greeting).to_have_text(f"Hello, {session.user.name}")
    products_page.card(DESK_LAMP.name).add_to_cart()
    products_page.card(COFFEE_MUG.name).add_to_cart()
    expect(products_page.header.cart_count).to_have_text("2")

    cart_page.open()
    expect(cart_page.items).to_have_count(2)
    expect(cart_page.total).to_have_text("₹1,498")
```

tests/cart/test_checkout.py

```python
import re

import pytest
from playwright.sync_api import expect

from framework.api.shop_api import ShopApi
from framework.data.products import WATER_BOTTLE
from framework.pages.cart_page import CartPage


@pytest.mark.cart
@pytest.mark.smoke
def test_places_an_order_for_a_cart_filled_through_the_api(session, api: ShopApi, cart_page: CartPage):
    api.add_to_cart(session.token, WATER_BOTTLE.id, 2)   # arrange through the API, test the UI
    cart_page.open()
    expect(cart_page.total).to_have_text("₹698")

    cart_page.place_order()
    expect(cart_page.status).to_have_text(re.compile(r"Order #\d+ placed"))
    expect(cart_page.header.cart_count).to_have_text("0")
```

Run it: `pytest -m smoke -v`

Lesson: Step 3: tests by feature, with tags

## TABLES

### Table: folder · python

| Folder | Python |
|---|---|
| framework code | framework/ |
| settings | framework/config.py |
| page objects | framework/pages/ |
| fixtures | conftest.py |
| tests | tests/<feature>/test_*.py |
| runner settings | pytest.ini |

Lesson: What a framework is, and the finished result

### Table: rule · why

| Rule | Why |
|---|---|
| settings come from the environment, with a local default | the same code runs on a laptop, in CI and against staging |
| secrets never go in the repository | passwords live in CI secrets and in a local .env that Git ignores |
| exact versions of every package | every machine, and every CI run, uses the same Playwright |

Lesson: Step 1: project, settings and runner config

### Table: layer · its one job · in this framework

| Layer | Its one job | In this framework |
|---|---|---|
| data | test users and products | users, products |
| API helper | fast set up without the browser | ShopApi: signIn, addToCart |
| components | parts on many pages | Header, ProductCard |
| page objects | one screen each | BasePage, LoginPage, ProductsPage, CartPage |
| fixtures | hand objects and a session to each test | test.extend, conftest.py, BaseTest |

Lesson: Step 2: data, API helpers, page objects and fixtures

### Table: pytest

| | pytest |
|---|---|
| mark a test | @pytest.mark.smoke or pytestmark |
| run only those | pytest -m smoke |
| leave them out | pytest -m 'not slow' |
| register the names | markers = in pytest.ini |

Lesson: Step 3: tests by feature, with tags

**Table: when you need · add · covered in**

| When you need | Add | Covered in |
|---|---|---|
| several environments | one .env per environment, or CI variables per job | this chapter, step 1 |
| many signed in roles | a setup step that saves storage state per role | API testing |
| a run on every pull request | a CI workflow with shards and a merged report | CI/CD |
| reports for managers and testers | Allure, pytest-html or ExtentReports next to the built-in reports | Reporting labs |
| visual and accessibility checks | toHaveScreenshot, aria snapshots and axe in their own folder | Visual, accessibility and time |
| lots of varied test data | small factory functions that build users and orders with unique names | Parallel runs |
| consistent code | a linter, a formatter and code review rules | your team |

Lesson: How the framework grows

### TIPS AND TRAPS

IN REAL TEAMS Commit a .env.example with the names of the settings and no values. A new team member copies it to .env and fills it in. The real .env is listed in .gitignore.

Lesson: Step 1: project, settings and runner config

REMEMBER The session fixture signs in through the API and puts the token in the page before it loads. Each test gets its own session, so its cart is its own, and parallel tests cannot disturb each other. This is the isolation rule from the parallel runs chapter, built into the framework.

Lesson: Step 2: data, API helpers, page objects and fixtures

IN REAL TEAMS Grow the framework when the tests ask for it. A layer nobody needs yet is code to maintain. The best frameworks are the ones where a new test takes ten lines and needs no new helper.

Lesson: How the framework grows

## 7.3 CI/CD pipelines

The framework ran with CI settings, a forgotten test.only failed the run, a wrong secret failed the sign in tests, shards merged into one report, and the smoke tests passed inside the official Docker images in all four languages.

### KEY IDEAS

- **Every push** Tests run by themselves. Nobody has to remember.
- **Fresh machine** Install everything each time. Or use the Docker image.
- **Secrets** From the CI settings. Masked in the logs.
- **Evidence** Reports kept, pass or fail. Traces explain failures.
- **Remember** CI runs the same three steps everywhere: a machine that can run browsers, the packages and browsers installed, then the tests with CI settings and secrets.

### CODE

**Run the suite the way the pipeline runs it**

pytest.ini

```
[pytest]
testpaths = tests
pythonpath = .
addopts = --browser chromium --tracing retain-on-failure --screenshot only-on-failure
markers =
    smoke: quick checks that run on every commit
    cart: the cart and checkout
```

Run it: `pytest -n 2 --junitxml=results/junit.xml && ls results`

Lesson: Running the suite the way the pipeline does

### The workflow file each project keeps in .github/workflows

.github/workflows/tests.yml

```yaml
name: Playwright tests
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: python -m playwright install --with-deps chromium
      - run: pytest -n 2 --junitxml=results/junit.xml
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
          name: test-results
          path: |
            results/
            test-results/
```

Run it: `git push`

Lesson: GitHub Actions, line by line

### What a run looks like when a secret is wrong

framework/data/users.py

```python
import os
from dataclasses import dataclass


@dataclass(frozen=True)
class User:
    email: str
    password: str
    name: str


# Test accounts. Real projects read passwords from CI secrets, never from the repository.
ASHA = User("asha@shop.example", os.getenv("ASHA_PASSWORD") or "s3cret", "Asha")
RAVI = User("ravi@shop.example", os.getenv("RAVI_PASSWORD") or "admin123", "Ravi")
```

Run it: `ASHA_PASSWORD=wrong pytest tests/smoke -q --tb=line`

Lesson: Secrets and settings per environment

### Each matrix job runs its own part of the suite

.github/workflows/tests.yml

```yaml
name: Playwright tests (split)
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
```

```
      matrix:
        group: [1, 2]
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: python -m playwright install --with-deps chromium
      - run: pytest --splits 2 --group ${{ matrix.group }} --junitxml=results/junit-${{ matrix.group }}.xml
        env:
          BASE_URL: ${{ vars.BASE_URL }}
          ASHA_PASSWORD: ${{ secrets.ASHA_PASSWORD }}
          RAVI_PASSWORD: ${{ secrets.RAVI_PASSWORD }}
      - uses: actions/upload-artifact@v5
        if: ${{ !cancelled() }}
        with:
          name: results-${{ matrix.group }}
          path: results/
```

Run it: `for i in 1 2; do echo "== job group $i of 2"; pytest --splits 2 --group $i -q; done`

Lesson: Shards with a matrix, and one merged report

### The framework inside the official Playwright image

`requirements.txt`

```
playwright==1.63.0
pytest==9.1.1
pytest-playwright==0.10.0
pytest-split==0.11.0
pytest-xdist==3.8.0
python-dotenv==1.2.4
```

Run it: `docker run --rm --init --ipc=host --network host -v "$PWD":/work -w /work mcr.microsoft.com/playwright/python:v1.63.0-noble /bin/bash -c "pip install -q -r requirements.txt && pytest -m smoke"`

Lesson: The official Docker images

### A Jenkinsfile for each project

`Jenkinsfile`

```
pipeline {
  agent {
    docker {
      image 'mcr.microsoft.com/playwright/python:v1.63.0-noble'
      args '-u root'                              // so pip may install into the image
    }
  }
  environment {
    BASE_URL = 'https://staging.shop.example'
    ASHA_PASSWORD = credentials('asha-password')
  }
  stages {
    stage('Install') {
      steps { sh 'pip install -r requirements.txt' }
    }
    stage('Test') {
      steps { sh 'pytest -n 2 --junitxml=results/junit.xml' }
    }
  }
  post {
    always {
      junit 'results/junit.xml'
      archiveArtifacts artifacts: 'test-results/**', allowEmptyArchive: true
    }
  }
}
```

Lesson: Jenkins, GitLab and Azure DevOps

## TABLES

### Table: on your laptop · on ci

| | On your laptop | On CI |
|---|---|---|
| the machine | yours, with everything already installed | a fresh machine for every run |
| the browser | installed once | installed in every run, or already in a Docker image |
| the settings | a local .env file | variables and secrets in the CI settings |
| a failure | you watch it happen | you read the report, the logs and the trace afterwards |
| test.only | handy while debugging | must fail the run |

Lesson: What CI does for your tests

### Table: setting on ci · why

| Setting on CI | Why |
|---|---|
| forbidOnly: true | a forgotten test.only stops the run instead of skipping the rest |
| retries: 2 | a test that fails once and then passes is reported as flaky, not hidden |
| a fixed number of workers | CI machines are smaller than laptops. The Playwright docs suggest 1 for the most stable runs, or sharding for speed |
| trace: on-first-retry | every retried test leaves a trace to open later |

Lesson: Running the suite the way the pipeline does

### Table: line · what it does

| Line | What it does |
|---|---|
| on: push / pull_request | when the workflow runs |
| runs-on: ubuntu-latest | a fresh Linux machine from GitHub |
| timeout-minutes: 30 | stops a job that hangs. Keep it above any global timeout in the test config |
| actions/checkout | gets the code of the commit |
| setup-node, setup-python, setup-java | installs the language version |
| npm ci, pip install -r, mvn | installs the exact package versions from the lock or requirements file |
| playwright install --with-deps chromium | downloads Chromium and installs the Linux libraries it needs |
| env: with secrets. and vars. | hands settings and passwords to the tests |
| if: ${{ !cancelled() }} | uploads the results even when tests failed, which is when you need them most |
| upload-artifact | keeps the report as a download on the run page |

Lesson: GitHub Actions, line by line

### Table: trigger · use it for

| Trigger | Use it for |
|---|---|
| push to main | checking what was merged |
| pull_request | checking a change before it is merged |
| schedule with cron | a nightly run of the full suite, for example in every browser |
| workflow_dispatch | a Run workflow button for runs by hand |
| deployment_status | testing a preview or staging deployment once it is ready |

Lesson: GitHub Actions, line by line

### Table: kind · in github · in the workflow · seen in logs

| Kind | In GitHub | In the workflow | Seen in logs |
|---|---|---|---|
| a setting such as BASE_URL | Settings, Secrets and variables, Actions, Variables | ${{ vars.BASE_URL }} | yes |
| a password | the same page, Secrets | ${{ secrets.ASHA_PASSWORD }} | no, GitHub masks it as *** |

Lesson: Secrets and settings per environment

### Table: pytest

| | pytest |
|---|---|
| split the suite | --splits 2 --group 1 (pytest-split) |
| balance | by stored durations with --store-durations |
| one report | one JUnit file per job |

Lesson: Shards with a matrix, and one merged report

### Table: language · image

| Language | Image |
|---|---|
| Python | mcr.microsoft.com/playwright/python:v1.63.0-noble |

Lesson: The official Docker images

### Table: flag · why the playwright docs recommend it

| Flag | Why the Playwright docs recommend it |
|---|---|
| --ipc=host | without it Chromium can run out of memory and crash |
| --init | avoids special treatment of the process with id 1, a common cause of leftover processes |
| a fixed tag such as v1.63.0-noble | the image must match the Playwright version in the project. noble means Ubuntu 24.04 |
| --user pwuser with a seccomp profile | only when the tests visit sites you do not trust |

Lesson: The official Docker images

### Table: ci service · file · sets ci=true · secrets

| CI service | File | Sets CI=true | Secrets |
|---|---|---|---|
| GitHub Actions | .github/workflows/*.yml | yes | secrets.NAME |
| GitLab CI | .gitlab-ci.yml | yes | CI/CD variables, marked masked |
| Jenkins | Jenkinsfile | no, set it yourself | credentials('id') |
| Azure Pipelines | azure-pipelines.yml | no, set it yourself | secret pipeline variables, $(NAME) |

Lesson: Jenkins, GitLab and Azure DevOps

### Table: step · pytest

| Step | pytest |
|---|---|
| 1. read the failure in the job log | the short summary at the end |
| 2. download the artifact | test-results |
| 3. open the trace | playwright show-trace trace.zip |
| 4. run only the failures again | pytest --lf |

Lesson: When the pipeline goes red

### TIPS AND TRAPS

**REMEMBER** The Playwright documentation sums CI up in three steps: make sure the machine can run browsers, install the packages and browsers, then run the tests. Everything in this chapter is a version of those three steps.

Lesson: What CI does for your tests

**IN REAL TEAMS** pytest and TestNG have no test.only, so there is nothing to forbid. In pytest the closest mistake is leaving `-k` or a single file in the CI command, so keep the command in the workflow file and review changes to it.

Lesson: Running the suite the way the pipeline does

**IN REAL TEAMS** In the repository settings, a branch rule can make this workflow a required check. Then a pull request cannot be merged while its tests are red.

Lesson: GitHub Actions, line by line

**COMMON MISTAKE** Secrets are not given to workflows that run for pull requests from forks, so a fork cannot read your passwords. Tests that need them are skipped or fail there, which is expected.

Lesson: Secrets and settings per environment

**REMEMBER** fail-fast: false keeps the other shards running when one fails, so you see every failure in one run. The merge job uses if: !cancelled() for the same reason.

Lesson: Shards with a matrix, and one merged report

**IN REAL TEAMS** In GitHub Actions, the same image goes under `container: image:` in the job, with `options: --user 1001`, and the browser install step is left out.

Lesson: The official Docker images

**COMMON MISTAKE** Never fix a red pipeline by adding a skip or a longer sleep. Find out whether the app or the test is wrong. The trace from the debugging chapter shows the page at every step of the failed run.

Lesson: When the pipeline goes red

**IN REAL TEAMS** A test marked flaky in the report passed on a retry. Treat it as a bug to fix this week, not as a pass.

Lesson: When the pipeline goes red

## 7.4 Reporting labs

One out of date test failed in every lab, and each report showed it: JUnit XML and JSON counts, the Playwright HTML report, pytest-html and ExtentReports with screenshots, Allure Report in all four languages, a custom summary, and one HTML report merged from two shards.

### KEY IDEAS

- **For you** list, dot, -v. Right now.
- **For CI** Counts and history. Every server reads it.
- **For testers** Error, screenshot, trace. Why it failed.
- **For everyone** Same report, any language. Trends and labels.
- **Remember** Write several reports from one run: a short one for the terminal, JUnit XML for CI, and an HTML report with evidence for people.

### CODE

**Lab 1: the reports a machine reads**

`tests/products/test_price.py`

```
from playwright.sync_api import expect

from framework.data.products import DESK_LAMP
from framework.pages.products_page import ProductsPage
```

```python
# Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
def test_the_desk_lamp_costs_1199(products_page: ProductsPage):
    products_page.open()
    expect(products_page.card(DESK_LAMP.name).price).to_have_text("₹1,199")
```

Run it: `pytest -q --tb=line --junitxml=results/junit.xml; grep -o '<testsuite [^>]*>' results/junit.xml`

Lesson: Lab 1: built-in reporters and JUnit XML

### Lab 2: an HTML report for people

tests/products/test_price.py

```python
from playwright.sync_api import expect

from framework.data.products import DESK_LAMP
from framework.pages.products_page import ProductsPage


# Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
def test_the_desk_lamp_costs_1199(products_page: ProductsPage):
    products_page.open()
    expect(products_page.card(DESK_LAMP.name).price).to_have_text("₹1,199")
```

pytest.ini

```ini
[pytest]
testpaths = tests
pythonpath = .
addopts = --browser chromium --tracing retain-on-failure --screenshot only-on-failure
markers =
    smoke: quick checks that run on every commit
    cart: the cart and checkout
```

Run it: `pytest -q --tb=line --html=reports/report.html --self-contained-html; ls reports`

Lesson: Lab 2: HTML reports with evidence

### Lab 3: Allure Report, the same report for every language

tests/products/test_price.py

```python
import allure
from playwright.sync_api import expect

from framework.data.products import DESK_LAMP
from framework.pages.products_page import ProductsPage


# Out of date on purpose: the lamp now costs ₹1,299, so the reports have a failure to show
@allure.feature("Prices")
@allure.severity(allure.severity_level.CRITICAL)
def test_the_desk_lamp_costs_1199(products_page: ProductsPage):
    with allure.step("open the products page"):
        products_page.open()
    with allure.step("check the lamp price"):
        expect(products_page.card(DESK_LAMP.name).price).to_have_text("₹1,199")
```

tests/conftest.py

```python
import allure
import pytest


# After each test: when it failed, attach a screenshot of the page to the Allure results.
# pytest-playwright already puts its own screenshot and trace into pytest-html.
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    page = item.funcargs.get("page")
    if report.when == "call" and report.failed and page:
        allure.attach(page.screenshot(), name="screenshot at failure",
attachment_type=allure.attachment_type.PNG)
```

Run it: `pytest -q --tb=line --alluredir=allure-results; npx allure generate allure-results && ls allure-report/index.html`

Lesson: Lab 3: Allure Report in all four languages

### Lab 4: write your own reporter

framework/reporting/summary.py

```
import pytest

results = []


# pytest reports each test three times: setup, call and teardown. The call holds the result,
# and a failure while setting up is reported in the setup instead.
def pytest_runtest_logreport(report):
    if report.when == "call" or (report.when == "setup" and not report.passed):
        results.append(report)


def pytest_terminal_summary(terminalreporter):
    tr = terminalreporter
    tr.section("shop summary")
    for r in results:
        name = r.nodeid.split("::")[-1]
        tr.write_line(f"{'PASS' if r.passed else 'FAIL'}  {name} ({r.duration:.1f} s)")
    failed = [r.nodeid.split("::")[-1] for r in results if r.failed]
    tr.write_line(f"{len(results) - len(failed)} passed, {len(failed)} failed")
    for name in failed:
        tr.write_line(f"  to look at: {name}")
```

Run it: `pytest -q --tb=no -p framework.reporting.summary; echo "exit code $?"`

Lesson: Lab 4: a custom reporter

## TABLES

### Table: reader · wants · report

| Reader | Wants | Report |
|---|---|---|
| you, at the terminal | pass or fail, right now | list, line or dot in Playwright Test, -v or -q in pytest, the Surefire summary in Maven |
| the CI server | counts and history | JUnit XML, which every CI server reads |
| a tester fixing a failure | the error, the screenshot, the trace | the Playwright HTML report, pytest-html, Allure, ExtentReports |
| a manager | a dashboard: how many passed, trends, which features | Allure, ExtentReports |
| your own tools | data in a known shape | JSON, or a custom reporter |

Lesson: What a test report is for

### Table: evidence · pytest

| Evidence | pytest |
|---|---|
| screenshot at failure | --screenshot only-on-failure |
| video | --video retain-on-failure |
| trace | --tracing retain-on-failure |
| your own file | allure.attach or a pytest-html extra |

Lesson: Lab 2: HTML reports with evidence

### Table: language · adapter · writes results with

| Language | Adapter | Writes results with |
|---|---|---|
| Python | allure-pytest (pip) | --alluredir=allure-results |

Lesson: Lab 3: Allure Report in all four languages

**Table: pytest**

| | pytest |
|---|---|
| at the start | pytest_sessionstart |
| after each test | pytest_runtest_logreport(report) |
| at the end | pytest_terminal_summary |
| turn it on | -p module, or code in conftest.py |

Lesson: Lab 4: a custom reporter

**Table: where · how**

| Where | How |
|---|---|
| GitHub Actions | upload the report folder as an artifact, or publish it to GitHub Pages |
| Jenkins | the junit step for counts and history, and the HTML Publisher or Allure plugin for the page |
| GitLab CI | artifacts: reports: junit, and the report folder as an artifact or GitLab Pages |
| Azure DevOps | PublishTestResults for JUnit XML, and PublishPipelineArtifact for the HTML |

Lesson: Lab 5: merging and publishing reports in CI

**Table: need · python**

| Need | Python |
|---|---|
| CI counts | --junitxml |
| debugging a failure | pytest-html with traces from pytest-playwright |
| one report for many languages and teams | Allure |
| something special | hooks in a plugin |

Lesson: Which report should a team use?

## TIPS AND TRAPS

REMEMBER The labs in this chapter all use the framework from the framework chapter, plus one test that fails on purpose: it still expects the lamp to cost ₹1,199 after a price change to ₹1,299. A report is only interesting when something fails.

Lesson: What a test report is for

IN REAL TEAMS The report is only as good as its evidence. A failure with a screenshot and a trace can often be understood in a minute, without running anything again.

Lesson: Lab 2: HTML reports with evidence

NEW IN PLAYWRIGHT This page uses Allure Report 3, which installs from npm and needs Node.js, even for Python and Java projects: `npx allure generate allure-results`. Both read the same allure-results folder.

Lesson: Lab 3: Allure Report in all four languages

IN REAL TEAMS Keep the allure-results of earlier runs, or the history folder of the last report, and Allure shows trends and flaky tests across runs.

Lesson: Lab 3: Allure Report in all four languages

COMMON MISTAKE A reporter must not change whether the run passed. It only describes the run, so CI still sees exit code 1 when a test failed.

Lesson: Lab 4: a custom reporter

REMEMBER

Python and Java shards each write their own JUnit XML file. CI servers read many files at once, for example target/surefire-reports/*.xml in Jenkins, so they do not need merging first. Allure also reads several allure-results folders in one command.

Lesson: Lab 5: merging and publishing reports in CI

## 7.5 What's new, and AI test agents

A 1.56 feature ran in all four languages, the agent definitions were created for real in the TypeScript framework, and a checklist shows how to keep AI-written tests honest.

### KEY IDEAS

- **Releases** New browsers each time. Read the notes first.
- **New APIs** In every language. Like consoleMessages.
- **Test Agents** Plan, generate, heal.
- **Playwright MCP** A browser for assistants. Any language.
- **Remember** Update on a branch and read the release notes. Let AI write tests, but review them like any other code, and make sure each one can fail.

### CODE

**New in 1.56: read console messages, errors and requests at any time**

tests/test_collect.py

```python
from playwright.sync_api import Page


def test_reads_console_messages_and_page_errors_after_the_fact(page: Page):
    page.goto("/widgets")
    page.get_by_role("button", name="Write to the console").click()
    with page.expect_event("pageerror"):
        page.get_by_role("button", name="Break the page").click()

    # New in 1.56: the page keeps its recent console messages, errors and requests, no listener needed
    for m in page.console_messages():
        print(f"console.{m.type}: {m.text}")
    for e in page.page_errors():
        print(f"page error: {e.message}")
    print(f"requests so far: {len(page.requests())}")
    assert len(page.page_errors()) == 1
```

Run it: `pytest -s -q`

Lesson: A new feature in all four languages

## TABLES

### Table: version · highlights for testers

| Version | Highlights for testers |
|---|---|
| 1.56 | Playwright Test Agents: planner, generator and healer. page.consoleMessages(), page.pageErrors() and page.requests() |
| 1.57 | a Speedboard tab in the HTML report that shows the slowest tests, and Chrome for Testing builds instead of Chromium |
| 1.58 | a timeline chart for merged reports, and old selectors such as _react removed |
| 1.59 | page.screencast for videos with chapters and overlays, page.ariaSnapshot(), and npx playwright trace on the command line |
| 1.60 | test.abort(), locator.drop() and HAR recording through tracing |
| 1.61 | passkeys through browserContext.credentials, page.localStorage, and -G as a short form of --grep-invert |
| 1.62 | retryStrategy, WebP screenshots, and npx playwright mcp bundled with Playwright |
| 1.63 | test locks, locator.visible(), --add-reporter and a perfetto reporter |
| 1.64 | page.webmcp, --shuffle, locator.within() and page.getByRef() |

Lesson: How Playwright changes, and how to keep up

### Table: language · update the package · then

| Language | Update the package | Then |
|---|---|---|
| Python | pip install -U playwright pytest-playwright | playwright install --with-deps |

Lesson: How Playwright changes, and how to keep up

### Table: agent · reads · writes

| Agent | Reads | Writes |
|---|---|---|
| planner | your running app, starting from the seed test | a Markdown plan in specs/ |
| generator | a plan from specs/ | test files in tests/, with each locator and check tried live |
| healer | a failing test | a fixed test, or a skipped one when it believes the app itself is broken |

Lesson: Playwright Test Agents

### Table: tool · use it for

| Tool | Use it for |
|---|---|
| Playwright MCP | any assistant that speaks MCP, exploring an app and writing tests in any language |
| Copy prompt in the HTML report, trace viewer and UI mode | pasting a failed test with its error and page snapshot into any assistant, since 1.51 |
| the Playwright CLI with skills | coding agents. The MCP README notes that the CLI uses fewer tokens |

Lesson: Playwright MCP: a browser for AI assistants

**Table: check · why**

| Check | Why |
|---|---|
| the test uses the framework: page objects, fixtures, test data | otherwise every generated test adds locators that someone must maintain |
| every check is meaningful | a test that only checks that a heading is visible passes even when the feature is broken |
| no fixed waits and no force: true | they hide timing problems that real users meet |
| the healer did not hide a bug | it may skip a test when the app looks broken. A skipped test needs a person to decide |
| the test fails when the feature breaks | break the feature on purpose once, or change an expected value, and watch it go red |
| secrets stay out of prompts and plans | plans and prompts are files and messages that other people and tools can read |

Lesson: Using AI-written tests safely

## TIPS AND TRAPS

IN REAL TEAMS Read the release notes before updating, and update on a branch so CI shows what changed. Each release lists its breaking changes, such as page.accessibility removed in 1.57 and Node.js 16 support removed in 1.54.

Lesson: How Playwright changes, and how to keep up

IN REAL TEAMS Use pageErrors() at the end of a test to fail it when the page threw an error that the user never saw. It is a cheap check that catches broken scripts early.

Lesson: A new feature in all four languages

COMMON MISTAKE The agents write Playwright Test code in TypeScript or JavaScript. For Python and Java there are no ready made agents, but the Playwright MCP server below lets an assistant drive the browser while it writes pytest or Java tests.

Lesson: Playwright Test Agents

IN REAL TEAMS Give the agents a seed test that signs in through your own fixtures. Generated tests then start from the same state as the rest of your suite.

Lesson: Using AI-written tests safely